Folium: Behavioral Decision Intelligence Through Calibrated Bias Detection, Causal Analysis, and Paired Long-Horizon Portfolio Simulation

submitted in partial fulfillment of the requirement
for the award of the Degree of

Bachelor of Technology
in
Computer Engineering

by

Jesil Shah ([UID No.])
Param Shah ([UID No.])
Het Shah ([UID No.])

under the guidance of

Dr. Nataasha Raul



Department of Computer Engineering
Bharatiya Vidya Bhavan's
Sardar Patel Institute of Technology
(Autonomous Institute Affiliated to University of Mumbai)
Munshi Nagar, Andheri-West, Mumbai-400058
University of Mumbai
[Month Year]

Certificate

This is to certify that the Project entitled "Folium: Behavioral Decision Intelligence Through Calibrated Bias Detection, Causal Analysis, and Paired Long-Horizon Portfolio Simulation" has been completed to our satisfaction by Mr. Jesil Shah, Mr. Param Shah and Mr. Het Shah under the guidance of Dr. Nataasha Raul for the award of Degree of Bachelor of Technology in Computer Engineering from University of Mumbai.

Certified by

Dr. Nataasha Raul
Project Guide

Dr. Sudhir Dhage
Head of Department

Dr. Bhalchandra N. Chaudhari
Principal

Department of Computer Engineering
Bharatiya Vidya Bhavan's
Sardar Patel Institute of Technology
(Autonomous Institute Affiliated to University of Mumbai)
Munshi Nagar, Andheri(W), Mumbai-400058
University of Mumbai
[Month Year]

Project Approval Certificate

This is to certify that the Project entitled "Folium: Behavioral Decision Intelligence Through Calibrated Bias Detection, Causal Analysis, and Paired Long-Horizon Portfolio Simulation" by Mr. Jesil Shah, Mr. Param Shah and Mr. Het Shah is found to be satisfactory and is approved for the award of Degree of Bachelor of Technology in Computer Engineering from University of Mumbai.

External Examiner **Internal Examiner**

(signature) (signature)

Name: **Name:** Prof. Jyotsna Bhagat

Date: **Date:**

Seal of the Institute

Statement by the Candidates

We wish to state that the work embodied in this thesis titled "Folium: Behavioral Decision Intelligence Through Calibrated Bias Detection, Causal Analysis, and Paired Long-Horizon Portfolio Simulation" forms our own contribution to the work carried out under the guidance of Dr. Nataasha Raul at the Sardar Patel Institute of Technology. We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission.

Name and Signature:

1. Jesil Shah

2. Param Shah

3. Het Shah

Acknowledgments

It is a pleasure to acknowledge the help and support that has gone into making this report. We express our sincere gratitude to our guide, Dr. Nataasha Raul, for the guidance and encouragement given throughout the project. The regular reviews, and the insistence that every number shown to an investor must be traceable to a computation, shaped many of the decisions described in this report.

We thank our internal examiner, Prof. Jyotsna Bhagat, for the advice given during the project reviews.

We thank the Head of Department, Dr. Sudhir Dhage, and the staff of the Department of Computer Engineering for giving us all the facilities required to carry out this work, and the Principal, Dr. Bhalchandra N. Chaudhari, for the support of the institute.

We would like to thank our family members and well wishers for their constant encouragement, without which we could not have completed this work.

Jesil Shah
Param Shah
Het Shah

Abstract

Decisions in retail investing are influenced not only by movements in the market but also by unrealized gains and losses, the liquidity of the household, the holding period and the reaction of the investor to uncertainty. Tools that predict prices can hardly tell whether a particular intended decision is the result of a behavioral pattern, or what the alternative course of action would mean for the investor's financial position in the future. In this project we propose Folium, an integrated decision support system and web application that evaluates one intended action (sell, hold or buy more) on one holding against the whole financial picture of the investor.

For every analysis, Folium reads the reason the investor gives in their own words for any stated need for cash, and then classifies the most likely behavioral pattern behind the decision (loss aversion, herd behavior, the disposition effect or a rational motive) using a calibrated XGBoost classifier with 22 features and exact TreeSHAP attributions. It compares the wealth of holding and of selling over the investor's own horizon on the same 1,000 block-bootstrapped market paths built from real NIFTY 50 and gold history, after Indian capital gains tax and charges. Cause and effect is estimated with DoWhy backdoor adjustment and g-computation. A fact-driven template engine explains the numbers and assumptions in English and Hindi, the verdict (HOLD, SELL or BUY MORE) comes from written evidence rules rather than a language model, and every analysis is sealed in a per-investor SHA-256 hash chain that can be verified later. The system is built with FastAPI, PostgreSQL with row-level security, Redis and React.

On a held-out test set of 4,000 synthetic decisions placed on real market days, the classifier achieves an accuracy of 87.9%, a macro-F1 score of 0.840 and an expected calibration error of 0.014. It is significantly better than a rules-only baseline and logistic regression, and statistically tied with a random forest. Among six market path generators, only the 12-month block bootstrap reproduces both the spread of real one-year returns and volatility clustering. The causal estimates recover the known generator effects with a mean absolute error of about 0.5 percentage points. These results are at the development stage and are obtained using synthetic behavioral labels instead of real investors' decisions.

Keywords: behavioral finance, decision support, XGBoost, probability calibration, SHAP, causal inference, block bootstrap, Monte Carlo simulation, explainable AI.

Contents

Acknowledgments i

Abstract ii

List of Abbreviations viii

1 Introduction 1
- 1.1 Problem Statement . . . 2
- 1.2 Motivation . . . 2
- 1.3 Objectives . . . 2
- 1.4 Scope . . . 3
- 1.5 Layout of the Report . . . 3

2 Literature Survey 4
- 2.1 Behavioral Foundations and Observed Investor Actions . . . 4
- 2.2 Machine Learning for Financial Classification . . . 5
- 2.3 Explainable Machine Learning . . . 6
- 2.4 Causal Modeling and Identification . . . 6
- 2.5 Long-Horizon Simulation and Portfolio Choice . . . 7
- 2.6 Security Foundations for a Financial Web Application . . . 7
- 2.7 Comparison of Existing Approaches . . . 8
- 2.8 Research Gaps . . . 8

3 Requirement Analysis 10
- 3.1 Functional Requirements . . . 11
- 3.2 Non-Functional Requirements . . . 11
- 3.3 Hardware and Software Requirements . . . 12
- 3.4 Feasibility Study . . . 13
- 3.5 Assumptions and Constraints . . . 13

4 System Design 14
- 4.1 Architecture . . . 14
- 4.2 Analysis Pipeline Components . . . 15
- 4.3 Data-Flow Diagrams . . . 16
- 4.4 Use Case Diagram . . . 17
- 4.5 Activity Diagram . . . 18
- 4.6 Sequence Diagram . . . 19
- 4.7 Entity-Relationship Diagram . . . 20

4.8 Deployment Diagram . . . 21
4.9 User Interface Design . . . 21

5 Methodology **22**
5.1 Data Sources . . . 22
5.2 Dataset v4.1 . . . 22
5.3 Feature Engineering . . . 25
5.4 Behavioral Classification . . . 26
5.5 Paired Hold versus Sell Simulation . . . 26
5.6 Causal Estimation . . . 28
5.7 Evidence-Based Recommendation . . . 28
5.8 Explanation . . . 29
5.9 Tamper-Evident Audit Chain . . . 29

6 Implementation **30**
6.1 Dataset Generator . . . 30
6.2 Bias Model . . . 31
6.3 Simulator . . . 32
6.4 Causal Module . . . 33
6.5 Recommendation and Both Sides . . . 34
6.6 Pipeline Orchestrator . . . 34
6.7 Explanation Engine . . . 35
6.8 Data Layer and Row-Level Security . . . 36
6.9 Audit Chain . . . 36
6.10 Authentication and Google Sign-in . . . 37
6.11 Account Deletion and Data Export . . . 38
6.12 Front End . . . 38
6.13 Error Handling and Edge Cases . . . 39

7 Results and Analysis **40**
7.1 Experimental Setup . . . 40
7.2 Behavioral Classification Results . . . 41
7.2.1 Baselines and Other Learners . . . 42
7.3 Simulator Results . . . 44
7.3.1 Realism and Choice of Generator . . . 44
7.3.2 Effect of the Horizon and Stability of the Numbers . . . 45
7.3.3 Results over 1,000 Investors and Sensitivity . . . 46
7.3.4 Comparison with Real History . . . 48
7.4 Cause and Effect Results . . . 49
7.5 Explanation Quality . . . 50
7.6 Application Screenshots . . . 50
7.7 Discussion . . . 52

8 Testing **54**
8.1 Test Suites . . . 54
8.2 Unit Testing . . . 55
8.3 Integration Testing . . . 55
8.4 System Testing . . . 56
8.5 Validation, Edge Cases and Performance . . . 57

8.6 Defects Found and Resolved 58

9 Conclusion and Future Scope **59**
9.1 Conclusion 59
9.2 Limitations 60
9.3 Future Scope 60

Bibliography **62**

10 Publications **67**

List of Figures

4.1 System architecture of Folium 14
4.2 Components involved in one analysis 15
4.3 Data-flow diagram, level 0 16
4.4 Data-flow diagram, level 1 (the analysis process) 16
4.5 Use case diagram 17
4.6 Activity diagram of a decision analysis 18
4.7 Sequence diagram of an analysis with live progress 19
4.8 Entity-relationship diagram of the main tables 20
4.9 Deployment diagram of the development set-up 21

5.1 Generation of one case in dataset v4.1 23
5.2 (a) Label distribution in dataset v4.1; (b) market regimes among real trading days and sampled cases 25

7.1 Confusion matrix (rows normalized, real frequency) 41
7.2 Reliability diagram after calibration 41
7.3 (a) Accuracy with 95% bootstrap confidence intervals; (b) mean and standard deviation over five split seeds 43
7.4 (a) Kolmogorov-Smirnov distance to real one-year returns; (b) volatility clustering 44
7.5 (a) Share of futures in which holding beats selling; (b) median paired gap, by horizon 45
7.6 Monte Carlo convergence for the base case (20 repeats per size) . . . 46
7.7 Share of investors in each verdict band 47
7.8 Share of verdicts that change across 300 investors when one assumption is changed 47
7.9 Company-specific volatility against the share of 20-year futures in which holding wins, for 24 NIFTY 50 stocks 48
7.10 Effect of each factor on the chance of selling: true effect against the g-computation estimate 49
7.11 Result page with the recommendation, both futures and the pattern found 50
7.12 Live five-step progress during an analysis 51
7.13 Six-part explanation in Hindi 51
7.14 Portfolio page with net worth, allocation and holdings 51
7.15 Verification of an audit record 52
7.16 Google sign-in return and the Welcome profile step 52

List of Tables

2.1 Comparison of existing approaches with Folium 8

3.1 Functional requirements 11
3.2 Hardware and software requirements 12

5.1 Label definitions used by the generator 24
5.2 Tax rules used by the simulator 27
5.3 Recommendation rules 28

6.1 Organization of the repository 30
6.2 Authentication and session controls 37
6.3 Edge cases and how they are handled 39

7.1 Hyperparameters of bias model v4.1 40
7.2 Overall test performance of the bias model 41
7.3 Per-class metrics (real frequency) 41
7.4 Confidence bands and how often each is right 42
7.5 Baselines, ablation and ceiling 42
7.6 Learners compared on the same features and guard (5 split seeds; bootstrap CIs and McNemar tests on seed 7) 43
7.7 Six market path generators against the real index (10,000 ten-year paths each, same historical mean) 44
7.8 Effect of the horizon (Rs. 1 lakh position; investor earning Rs. 18 lakh a year with Rs. 11 lakh liquid) 45
7.9 Paired simulation over 1,000 investors drawn from dataset v4.1 46
7.10 Walk-forward replay of a fund up 30% through every real window . . 48
7.11 Average effects of each factor, true and estimated 49

8.1 Automated test suites (635 tests in total) 54
8.2 Representative unit tests 55
8.3 Integration test scenarios 56
8.4 System test cases 57
8.5 Defects and their status 58

9.1 Achievement of the objectives 59
9.2 Future scope by phase 61

List of Abbreviations

API	Application Programming Interface
ATE	Average Treatment Effect
CI	Confidence Interval
CSRF	Cross-Site Request Forgery
DFD	Data-Flow Diagram
ECE	Expected Calibration Error
EMI	Equated Monthly Instalment
ER	Entity-Relationship
FD	Fixed Deposit
GBM	Geometric Brownian Motion
JSON	JavaScript Object Notation
JWT	JSON Web Token
KS	Kolmogorov-Smirnov
LTCG	Long-Term Capital Gains
MFA	Multi-Factor Authentication
NIFTY 50	National Stock Exchange of India 50-stock index
OIDC	OpenID Connect
RLS	Row-Level Security
SHAP	SHapley Additive exPlanations
SIP	Systematic Investment Plan
SPA	Single-Page Application
SSE	Server-Sent Events
STCG	Short-Term Capital Gains
TOTP	Time-based One-Time Password
VIX	Volatility Index (India VIX)
XGBoost	eXtreme Gradient Boosting

Chapter 1

Introduction

Participation of individual investors in Indian equity markets has grown sharply over the last decade. Discount brokers, mobile trading applications and the systematic investment plan (SIP) route into mutual funds have made it possible for a salaried person to own shares with a few taps on a phone. Access, however, is not the same as judgement. A first-time investor who watches a holding fall by 15% while friends discuss a market crash on a messaging group is facing a decision that is as much emotional as it is financial.

Risk perception, reference dependence, social information and personal constraints all influence such decisions. Prospect theory shows that people judge outcomes relative to a reference point and react more strongly to losses than to gains of the same size [1]. Empirical studies of trading records find a systematic tendency to realize gains early and hold on to losses [2], and investors are known to follow the crowd even when their own information points the other way [3]. These patterns are not rare quirks. They are systematic, and they cost money [14].

At the same time, the same action can be perfectly sensible for a different person. Selling a losing share is a mistake if it is driven by panic, but it is the right decision if the investor's college fees are due next month and there is no emergency fund. A tool that labels a decision as biased without looking at the investor's own finances will be wrong exactly when it matters most. This makes it necessary to assess the decision being made by the investor, rather than to make yet another prediction about the market.

Folium was built to address this gap. For a single holding and an intended action, it looks at the investor profile, the position and the market conditions at that moment, and tries to answer four distinct questions. What behavior pattern is shown by the observed characteristics? Which inputs determined that reading? What effects can be measured with a causal model? And what distribution of outcomes follows from each alternative under a stated market generator? These questions are not substitutes for one another. A high classification probability is not a psychological assessment, and a simulated wealth advantage is not a guarantee of success.

The project started as a research effort on behavioral bias detection with a Streamlit prototype and synthetic data. Over successive versions it grew into a complete web application with a FastAPI back end, a React front end, a PostgreSQL database

with row-level security, a dataset built on real NIFTY 50 and India VIX history, a calibrated classifier, a tax-aware simulator, a causal analysis module, a bilingual explanation engine and a tamper-evident audit trail.

1.1 Problem Statement

To design and implement a secure and explainable decision support system that, for one intended action (sell, hold or buy more) on one holding of an Indian retail investor, identifies the behavioral pattern most likely behind the decision with a calibrated confidence, compares the after-tax outcome of holding and selling over the investor's own horizon under the same simulated market conditions, estimates which factors push investors towards that action, explains the result in plain English or Hindi without allowing the wording to change any number, issues a recommendation from transparent rules, and records each analysis in a form that the investor can later verify.

1.2 Motivation

The cost of a behavioral mistake is real but invisible at the moment of decision. An investor who panic-sells during a correction rarely sees what the money would have become had it stayed invested. Folium makes that counterfactual concrete by projecting both choices on identical market scenarios.

Many financial assistants that use artificial intelligence produce fluent text whose numbers cannot be traced back to any calculation. For a decision about real money, a single invented figure is unacceptable, so we wanted a system in which every number on the screen comes from a deterministic computation and every assumption is disclosed. The Indian context also matters. Short and long-term capital gains tax, the importance of gold and fixed deposits in household portfolios, EMIs and education fees as fixed obligations, and the need for explanations in Hindi are rarely handled by tools designed elsewhere.

1.3 Objectives

Our key objectives for building this system are as follows:

- Design and implement a causal decision-time framework that detects behavioral biases and models cause-and-effect relationships beyond retrospective correlational analysis.
- Develop a personalized portfolio-level simulation engine to evaluate the long-term (10–30 years) impact of financial decisions under uncertainty and compounding effects.
- Generate decision-centric, counterfactual explanations that clearly show how financial decisions causally influence long-term outcomes beyond static post-hoc interpretations.

- Implement a tamper-proof audit mechanism that records AI recommendations, causal assumptions, and counterfactual analyses to ensure transparency, accountability, and verifiable decision trails.

1.4 Scope

The subject of a decision is a listed Indian share (NSE or BSE) or an equity mutual fund. Gold, fixed deposits, savings, property, loans, rent received and SIPs are used as context. Three actions (sell, hold and buy more), four behavioral classes and horizons of 1 to 30 years are supported, and the output is available in English and Hindi through a web interface that also works on phones.

Trade execution and broker integration, real-time trading signals, whole-portfolio optimization (each analysis concerns one position), derivatives, cryptocurrency and insurance, and inflation-adjusted projections are outside the scope. The application states clearly that its output is for information and education and is not investment advice. The intended users are retail investors who want a second opinion on a pending decision. An administrator role exists for resetting a user's two-step verification after a verified recovery request.

1.5 Layout of the Report

A brief overview of each chapter is provided here.
Chapter 2: A literature survey of behavioral finance, machine learning, explainability, causal modeling and simulation is presented.
Chapter 3: The functional and non-functional requirements and the feasibility of the system are described.
Chapter 4: The design of the system is explained with architecture, data-flow, use case, activity, sequence, entity-relationship and deployment diagrams.
Chapter 5: The methodology is explained, including the dataset and the mathematics of each model.
Chapter 6: The implementation of each module is described with selected code.
Chapter 7: The experimental evaluation and the results are presented and discussed.
Chapter 8: The testing carried out on the system is described.
Chapter 9: Conclusions and future scope.
Chapter 10: Publications.

Chapter 2

Literature Survey

This chapter reviews the theory and earlier work on which Folium is built. For each piece of work we note what it establishes, what it does not, and how it is used in the project.

2.1 Behavioral Foundations and Observed Investor Actions

Prospect Theory by Kahneman and Tversky [1] seeks to explain departures from the predictions of expected utility in situations of risk. Reference-dependent valuation, an asymmetric reaction to gains and losses, and non-linear probability weighting are its key features. It explains why an investor may behave as if a decision depends on the loss relative to the purchase price, even when the investment should be valued prospectively. It is a descriptive theory of choice and not a procedure for classifying a particular live situation. For Folium it gives the rationale to look at the unrealized gain or loss and the risk context as possible signals, but it does not give grounds for classifying every loss-making sale as irrational. Cumulative prospect theory [17] later added cumulative probability weighting and showed how choice under uncertainty differs from maximizing the expected value, which stresses the importance of reporting the downside tail instead of only the average simulated wealth.

In Odean's work [2], brokerage trading records are analyzed to see whether investors realize their winning stocks more readily than their losing ones. The analysis links behavioral finance theory to actual transactions and shows that the disposition effect can be measured from trading records rather than from self-reports alone. The study does not offer a decision-time classifier, a personalized intervention or a multi-decade counterfactual simulator. Folium uses the return and the context of a position as model features; without full trading records, a single observed decision is only suggestive of the disposition effect. Shefrin and Statman [4] connect the pattern of selling winners and holding losers to mental accounting, regret aversion and self-control, and Thaler's mental accounting [18] explains how people place gains and losses in separate mental accounts. These models explain why identical wealth situations can mean different things to different investors, but they do not provide a

mechanism for building a multi-class classifier that can be tested on market paths. Folium applies some of these ideas in a measurable form through portfolio features.

Bikhchandani, Hirshleifer and Welch [3] show how sequential choice can lead to informational cascades, in which people rationally adopt what others did earlier even when their own private information points the other way. Following the crowd is therefore not always irrational, since social signals carry some information. The model shows when cascades appear, but it cannot recover an individual's present intentions from market data. De Long et al. [19] show that noise traders can move prices even when rational arbitrageurs are present, so crowd measures are relevant to the market without being proof that a particular investor is herding. Folium therefore uses social indicators as proxies for herd behavior and not as conclusive evidence. Fama's review of efficient markets [20] provides the necessary balance: trading in response to information can be rational, so the system must allow rational readings instead of labelling every market-driven trade as a bias.

Barber and Odean [14] show that individual investors who trade more earn lower net returns, and in a separate study [15] they find that overconfidence shows up as trading intensity in a comparison by gender. Group-level patterns of this kind cannot serve as an individual diagnosis, so Folium avoids demographic shortcuts. Grinblatt and Keloharju [16] find that investor traits, past performance and the trading situation all affect behavior together, which also limits Folium: one questionnaire and one snapshot of the market can never capture everything.

2.2 Machine Learning for Financial Classification

Friedman [26] describes gradient boosting as the stage-wise optimization of a function. Chen and Guestrin [5] introduce XGBoost, a gradient-boosted tree framework with regularization, sparsity-aware split finding and scalable approximate learning. Its strength on tabular financial data is the ability to capture non-linear relationships between unrealized return, market volatility, liquidity and investment horizon without assuming a linear decision boundary. Random forests, introduced by Breiman [25] as ensembles of randomized decision trees, are a strong non-linear benchmark against which XGBoost can be compared on the same data splits, and the literature does not require boosted trees to outperform forests on artificial behavioral labels. Folium applies this family of models to a multi-class problem in which label quality and probability calibration matter as much as accuracy. Winning against synthetic rules may only show that the model has recovered the labelling process.

Zadrozny and Elkan [6] study how classifier scores can be turned into class probabilities. This matters because when a model shown to a user expresses certainty, accuracy alone cannot tell whether the stated probabilities match how often it is right. Niculescu-Mizil and Caruana [27] show that discrimination and probability estimation are separate problems, and that boosted trees in particular push probabilities away from 0 and 1. Guo et al. [28] show that high accuracy does not imply good calibration and introduce the expected calibration error; their experiments are mostly on neural networks, so no calibration method can be assumed optimal for a tree ensemble. Folium reports calibration error and reliability on held-out data. Calibration only checks agreement with the reference labels; it does not turn an

artificial behavioral label into reality.

Saito and Rehmsmeier [29] show that precision-recall analysis is more informative than ROC analysis under class imbalance. Folium is multi-class, but the same sensitivity to prevalence supports one-vs-rest precision and recall with macro-averaging, since relying on accuracy alone can hide problems with the minority class. López de Prado [24] discusses evaluation practices for financial machine learning, including the temporal nature of data, leakage and the problems of backtesting. Random splits of artificial data can overestimate generalization, so Folium also uses a temporal holdout.

2.3 Explainable Machine Learning

In the SHAP approach of Lundberg and Lee [7], a prediction is attributed to the features relative to a background distribution, and Lundberg et al. later gave an exact and fast algorithm for tree ensembles [35]. In Folium, SHAP shows whether unrealized loss, crowd or liquidity factors drove the behavioral reading. Its limitation is that SHAP explains the prediction function, not what would happen if a feature were changed in the real world, and the results depend on correlations between inputs. Janzing et al. [31] frame feature relevance as a causal problem and stress the difference between an observational contribution and an interventional effect. Folium therefore presents SHAP only as predictive attribution, while causal claims require a graph.

Ribeiro et al. [8] propose LIME, which approximates a black-box model locally with a simpler surrogate. Its faithfulness depends on how perturbations are sampled and how the neighborhood is built. Folium uses tree-specific attributions and so avoids fitting a local surrogate, but its explanations must still be checked numerically and linguistically. Counterfactual explanations by Wachter, Mittelstadt and Russell [9] specify input changes that would change a prediction, and Mothilal, Sharma and Tan [30] generate diverse counterfactuals. A change in a prediction is not necessarily an actionable change or a causal effect. Folium's HOLD versus SELL analysis is different: it compares two scenarios on paired market paths and answers a model-specific "what if" question, not a full causal question about the investor's real future. Weber, Carl and Hinz [34] review explainable AI in finance and stress evaluation that is tailored to the financial context. Folium adds bilingual, fact-checked explanation as a design criterion, while recognizing that readability and completeness alone do not prove that explanations improve decisions.

2.4 Causal Modeling and Identification

Pearl [10] develops structural causal models with directed acyclic graphs, interventions and adjustment criteria. The key lesson is that an observed association is not identified as a causal effect without assumptions about confounding and the graph. Folium models proposed relations between the market state, the portfolio, the investor's constraints and the decision with a directed graph, and its estimates depend on that graph and on the observed covariates; they should never be called causal truth. Robins introduced g-computation [36], and Hernán and Robins [37]

give a modern treatment of the identification assumptions of consistency, positivity and conditional exchangeability.

DoWhy, by Sharma and Kiciman [11], organizes causal analysis into four stages: modeling the graph, identifying the estimand, estimating, and refuting. Placebo tests, random common causes and subsampling can reveal instability, but passing them does not rule out every unobserved confounder, which is especially relevant for synthetic investor data. Chernozhukov et al. [32] provide double machine learning for treatment effects with flexible nuisance models; identification still depends on conditional exchangeability, so Folium's synthetic causal checks are not a replacement for a proper identification design.

2.5 Long-Horizon Simulation and Portfolio Choice

Künsch [12] develops resampling methods for dependent data, because independent bootstrap samples destroy temporal dependence. This is the reason for sampling contiguous blocks of returns when building market scenarios: block sampling better preserves volatility clustering and serial dependence than drawing independent months, although its accuracy still depends on the historical window, stationarity, the block length and how similar future regimes are to past ones. Politis and Romano's stationary bootstrap [33] uses random block lengths and is an alternative to fixed blocks. Efron and Tibshirani [13] formalize the bootstrap for measuring uncertainty, which supports reporting quantiles instead of a single deterministic figure. Folium reports the lower tail, median and upper tail of simulated wealth; these percentiles describe the simulator, not the coverage probabilities of future markets. Hamilton's regime-switching model [23] reminds us that returns are not drawn from one stable distribution, so sensitivity analysis must allow for changes of regime.

Markowitz [21] formulates portfolio selection through expected returns, variance and diversification, which suggests that a choice should be judged against the risk of the whole portfolio and not one investment alone. Merton [22] develops lifetime portfolio selection under uncertainty and shows the importance of horizon, consumption and changing wealth constraints. Folium takes these into account through scenario simulation, but it does not solve the optimal control problem.

2.6 Security Foundations for a Financial Web Application

Folium handles sensitive financial data, so the application layer is built on established standards. JSON Web Tokens are defined in RFC 7519 [38], and two-step verification uses time-based one-time passwords as in RFC 6238 [39]. Passwords are hashed with bcrypt [40] and their strength is estimated with zxcvbn [41]. Cross-site request forgery is mitigated with the double-submit cookie pattern described by OWASP [42], and per-user isolation uses PostgreSQL row-level security policies [43]. The tamper-evident audit log follows the hash-chaining idea of Haber and Stornetta [44] with SHA-256 [45], and live progress uses server-sent events [46].

2.7 Comparison of Existing Approaches

Table 2.1 compares the main kinds of tools available to a retail investor with the approach taken in this project.

Approach	**Looks at the decision?**	**Uses investor's finances?**	**Weakness for this problem**
Brokerage and portfolio trackers	No	Holdings only	Describe the asset, not whether the action is sensible
Robo-advisers	At onboarding	Risk questionnaire	Do not evaluate an individual pending trade
Behavioral questionnaires	No	Self-reported	Measure a trait, not the present situation
General chat assistants	Yes	Only if typed in	Numbers can be invented; no calibration or audit
Academic bias studies [2, 14]	After the fact	Account data	Population-level; not a decision-time tool
Folium	Yes, at decision time	Income, outflows, fund, portfolio, horizon	Behavioral labels are synthetic (Section 9.2)

Table 2.1: Comparison of existing approaches with Folium

2.8 Research Gaps

The reviewed literature in behavioral finance, portfolio theory, calibrated machine learning, causal identification, time-series resampling and explainable AI offers different but complementary methods. Five gaps emerge from it. First, tools and studies tend to treat any loss-making sale or early sale of a winner as a bias, whereas a decision-time tool has to recognize forced sales, rebalancing and near goals as rational. Second, classifiers shown to end users rarely report whether their confidence can be trusted. Third, feature attributions are often presented as causes. Fourth, simple "what if" calculators compare actions on different random scenarios or ignore tax, which adds noise and overstates the case for holding. Fifth, generated explanations may misstate figures, and checks that reject wording containing numbers absent from the computed facts are rare.

Behavioral patterns can only be used at the moment of decision if labelling is representative and exceptions are considered. Gradient boosting allows non-linear classification, but reliability and construct validity must be handled separately. Feature attribution improves interpretability without giving causal identification, defensible causal analysis needs graphs and sensitivity analysis, long-horizon resampling must account for temporal dependence and shifts in distribution, and a fluent explanation is not necessarily faithful to the simulation behind it. Folium combines candidate behavior identification, explicit bounds, paired decision comparison and fact-backed

explanation in a single decision-time process. This combination is the technical and evaluative contribution of the work; none of the individual algorithms is claimed to be newly invented.

Chapter 3

Requirement Analysis

The requirements below were drawn from the implemented code, the API routes, the database migrations and the project's master reference document. The status column shows whether a requirement is implemented and tested in the current release or planned for the next one.

3.1 Functional Requirements

ID	Requirement	Status
FR-1	Register with e-mail and password, verify the address, log in with an optional "keep me signed in", refresh sessions and log out	Built
FR-2	Reset a forgotten password through an e-mailed link	Built
FR-3	Two-step verification with an authenticator app (TOTP), eight single-use backup codes and an administrator-assisted recovery path	Built
FR-4	Sign in with Google end to end, including two-step verification, a grace-period account and a profile step for new accounts	Built
FR-5	Record age, take-home income, EMIs, rent, education fees, emergency fund, risk tolerance (five-question quiz) and horizon; re-verify before sensitive changes and log every change	Built
FR-6	Add, edit and remove shares (live prices) and mutual funds (searched and priced live); record gold, deposits, savings, property, loans, rent received and monthly SIP	Built
FR-7	Compute net worth, liquid assets, monthly surplus and allocation; fall back to average cost when a price is unavailable	Built
FR-8	Submit an analysis for one holding with an action, optional months held and a reason in the investor's own words	Built
FR-9	Capture and store the market context, portfolio snapshot and the stock's beta and own volatility at submission	Built
FR-10	Run the five-step analysis in the background and stream live progress, replaying it after a page reload	Built
FR-11	Read the comment for stated cash needs and feelings (English, Hinglish, Hindi) and quote them back verbatim	Built
FR-12	Show the behavioral pattern with High, Medium or Low confidence and the inputs that weighed most	Built
FR-13	Project HOLD and SELL over the investor's horizon after tax, with median, bad-case and good-case values by year	Built
FR-14	Show cause-and-effect factors and the single most helpful habit	Built
FR-15	Six-part explanation in English and Hindi; HTML report in both languages and PDF report in English	Built
FR-16	HOLD, SELL or BUY MORE verdict with its reasons and the numbers used, followed by the case for both sides	Built
FR-17	Seal each analysis in a tamper-evident chain and let the investor verify it	Built
FR-18	History of analyses with filtering by pattern and search by holding	Built
FR-19	Download all personal data; delete the account with a 30-day grace period; opt in to optional AI help	Built
FR-20	Administrator console with usage statistics, user management and a recovery queue	Planned (next release)

Table 3.1: Functional requirements

3.2 Non-Functional Requirements

Correctness and transparency were treated as the most important qualities. Every number shown to the investor has to come from a computation: explanation templates are authoritative, and optional AI wording is rejected if it introduces a number. Any input that is missing is filled with a disclosed default and listed on the result page under "How this was worked out".

Security and privacy follow from the sensitivity of the data. Passwords are hashed with bcrypt at cost 12, sessions use RS256 JSON Web Tokens (15-minute access and 7-day refresh tokens, rotated) in httpOnly cookies, mutating requests need a CSRF token, and two-step secrets are encrypted with Fernet. Every user-owned table is protected by PostgreSQL row-level security, and the application connects as a non-superuser role so that the policies apply. The audit table is append-only. Abuse is limited by rate limits of 5 logins per minute, 3 registrations and 3 password resets per hour, and 10 analyses per minute.

The system must also stay usable when something fails. A price or market-data outage never breaks a page; defaults are used and disclosed, and an explanation failure falls back to the template text. For performance, one simulation of 1,000 paths over 30 years takes about 0.1 seconds, and the analysis runs in the background so that the request returns immediately. For reproducibility, each decision stores its market and portfolio snapshot, and the dataset, training and simulation use fixed random seeds. For maintainability, the product reaches the research code through a single bridge module, and model versions can be rolled back with configuration switches.

3.3 Hardware and Software Requirements

Component	**Requirement**
Processor and memory	4-core x86-64 processor; 8 GB RAM (16 GB recommended), since PostgreSQL, Redis, the API, the front-end server and the ML libraries (about 400 MB on import) run together
Storage and network	About 3 GB of free space; internet access for live prices and optional services
Runtimes	Python 3.13 (required by the pinned scipy and DoWhy versions); Node.js with npm
Back end	FastAPI 0.115, Pydantic 2, asyncpg, Alembic, structlog, slowapi, Uvicorn
Authentication	PyJWT, bcrypt, zxcvbn, pyotp, Authlib, cryptography (Fernet)
Data stores	PostgreSQL 18; Redis-compatible store (Memurai on Windows)
Machine learning	numpy 2.4.2, pandas 3.0.1, scipy 1.17.1, scikit-learn 1.8.0, xgboost 3.2.0, shap 0.51.0, dowhy 0.14, yfinance 0.2.66
Explanation and reports	Jinja2 3.1.6, textstat, ReportLab; optional google-genai
Front end	React 19, Vite 8, TypeScript, Tailwind 4, TanStack Query, React Hook Form with Zod, Recharts, Vitest

Table 3.2: Hardware and software requirements

Python 3.13 is a hard requirement. DoWhy 0.14 restricts scipy to version 1.15.3 or lower on older Python versions, which would silently change the scipy version under which the model artefacts were built.

3.4 Feasibility Study

The project is technically feasible because every component relies on mature open-source libraries. XGBoost, scikit-learn and SHAP are standard for tabular modeling, DoWhy provides identification and refutation, FastAPI and asyncpg handle long-running work asynchronously, and PostgreSQL supports row-level security natively. The heavy computation (training the classifier in about 81 seconds and the causal study in about 54 seconds) happens offline, while one prediction and one simulation at request time are fast enough to run in a background task.

It is operationally feasible because the investor supplies information they already know (income, loans, holdings) once, and each analysis then needs only a holding, an action and an optional sentence. Gaps are filled with disclosed defaults instead of blocking the user, explanations avoid jargon and are available in Hindi. It is economically feasible because all the software is free and open source, market data comes from free sources, and the optional e-mail and Gemini services have free tiers and are off by default.

3.5 Assumptions and Constraints

Each analysis concerns one position. Horizons are capped at 30 years, with 20 years used when none is given. Tax rules are those of FY 2025-26 under the new regime, mutual funds are treated as equity funds, gold is valued at the international price before import duty and GST, and all amounts are nominal rupees. The profile records take-home pay, so where the simulator needs income before tax it inverts the income-tax formula. The models assume long-run growth of 11% for shares, 9% for gold, 6.75% for fixed deposits and 3% for savings, sale charges of 0.15%, and that a habit halves the tendency it acts on. These assumptions are stated to the investor on every result page.

Chapter 4

System Design

All diagrams in this chapter were drawn from the structure of the code in the repository.

4.1 Architecture

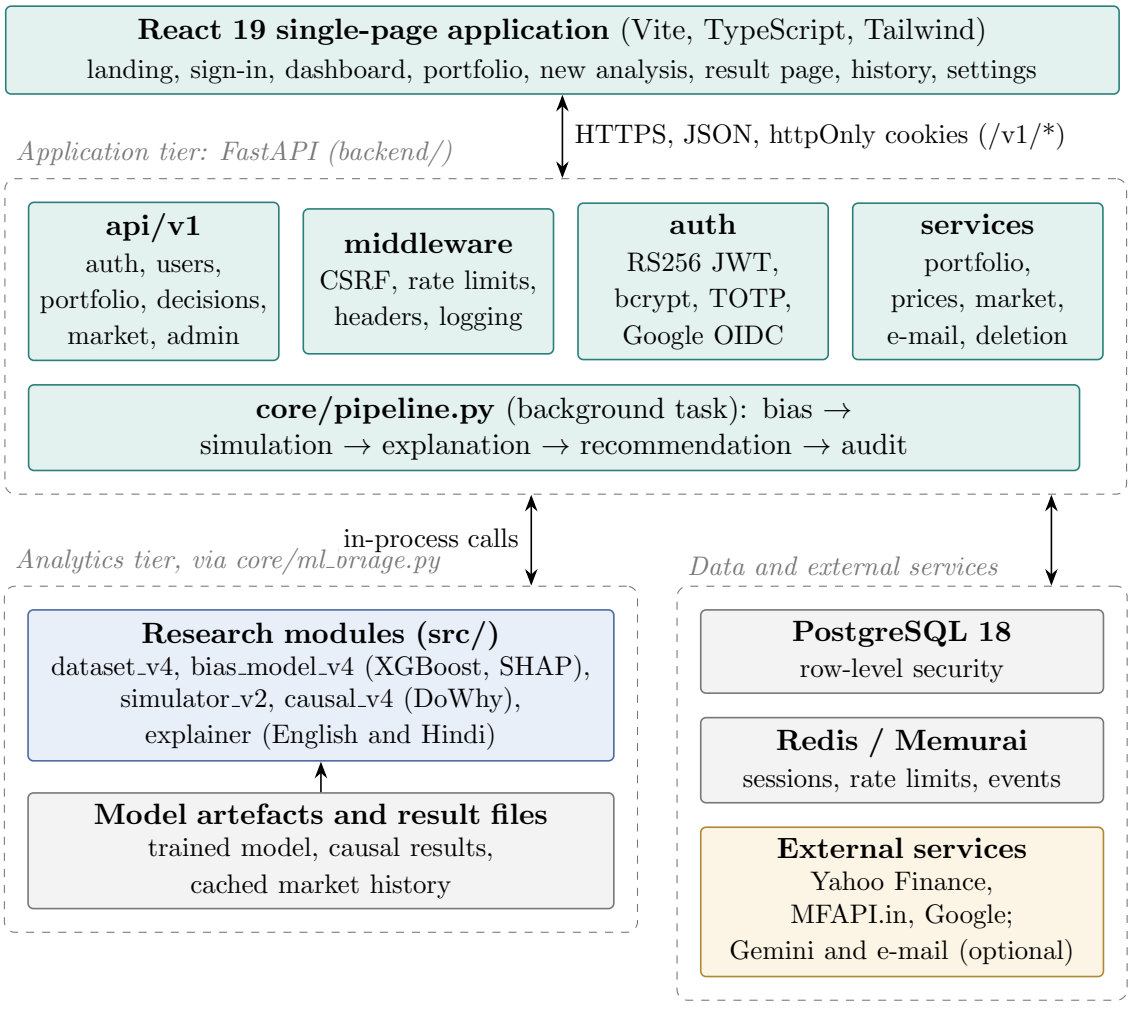


Figure 4.1: System architecture of Folium

Figure 4.1 shows the overall architecture. The presentation tier is a React single-page application. During development the Vite server forwards every `/v1/*` request to the API, so the browser sees a single origin, which keeps the httpOnly session cookies and the CSRF check simple.

The application tier is a FastAPI service. Its routes are in `backend/api/v1/`, its cross-cutting checks in `backend/middleware/`, its authentication code in `backend/auth/` and its domain services in `backend/services/`. The analysis itself runs as a background task under `backend/core/pipeline.py`.

The analytics tier is the research code in `src/`. An important design decision is that the back end never re-implements the models. The module `backend/core/ml_bridge.py` is the only place where the back end reaches into `src/`. It imports the heavy ML libraries lazily (they take about two seconds and 400 MB to load), runs blocking work in a worker thread so that the server stays responsive, and maps the vocabulary between the two sides. Since the research modules produced the reported numbers, re-implementing them would risk silently changing those numbers.

PostgreSQL stores users, portfolios, decisions and the audit chain under row-level security. A Redis-compatible store holds the token blacklist, the rate-limit counters and the event backlog for live progress. External services are called only for prices and market data, for Google sign-in, and optionally for e-mail and Gemini.

4.2 Analysis Pipeline Components

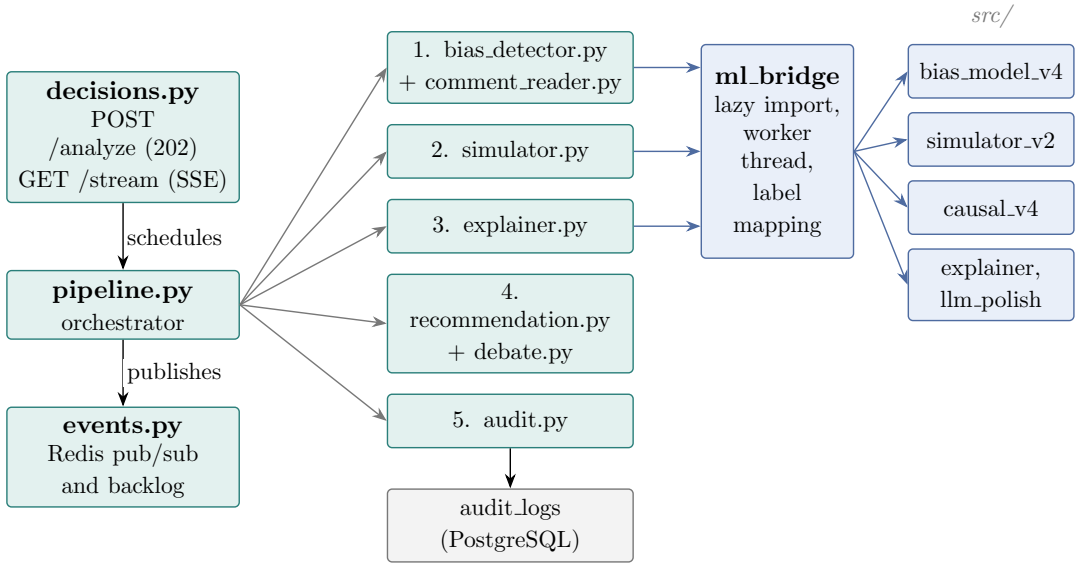


Figure 4.2: Components involved in one analysis

Figure 4.2 shows the components of one analysis. The analyze route (`POST` on `/v1/decisions/analyze`) schedules the pipeline as a background task and returns immediately. The orchestrator calls five steps in a fixed order: bias detection, simulation, explanation, recommendation and audit. Each step returns the columns it wants to update and the fields to attach to its progress event, while the orchestrator

alone owns the status, the events and persistence. It refuses any step that tries to write the columns it reserves for itself. The first three steps call the research code through the bridge, and the audit step writes to the `audit_logs` table.

4.3 Data-Flow Diagrams

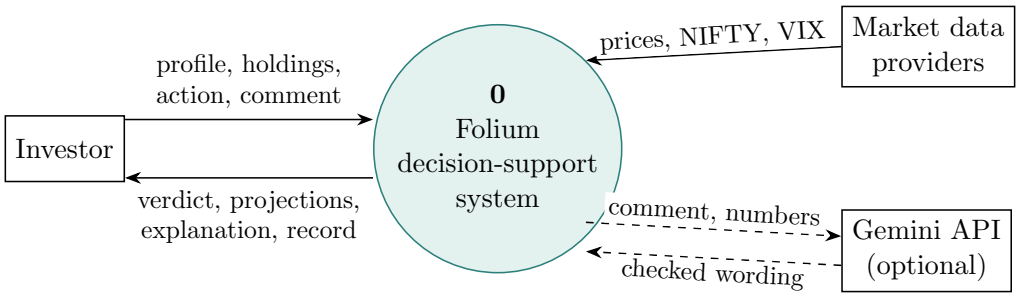


Figure 4.3: Data-flow diagram, level 0

At the context level (Figure 4.3) the system exchanges data with three external entities. The investor supplies a profile, holdings, the intended action and an optional comment, and receives a verdict, projections, an explanation and a verifiable record. Market data providers supply prices, the NIFTY index and the VIX. The Gemini API is optional: it receives a comment and the analysis numbers only when the investor has consented and the server has enabled it, and its output is accepted only after checks.

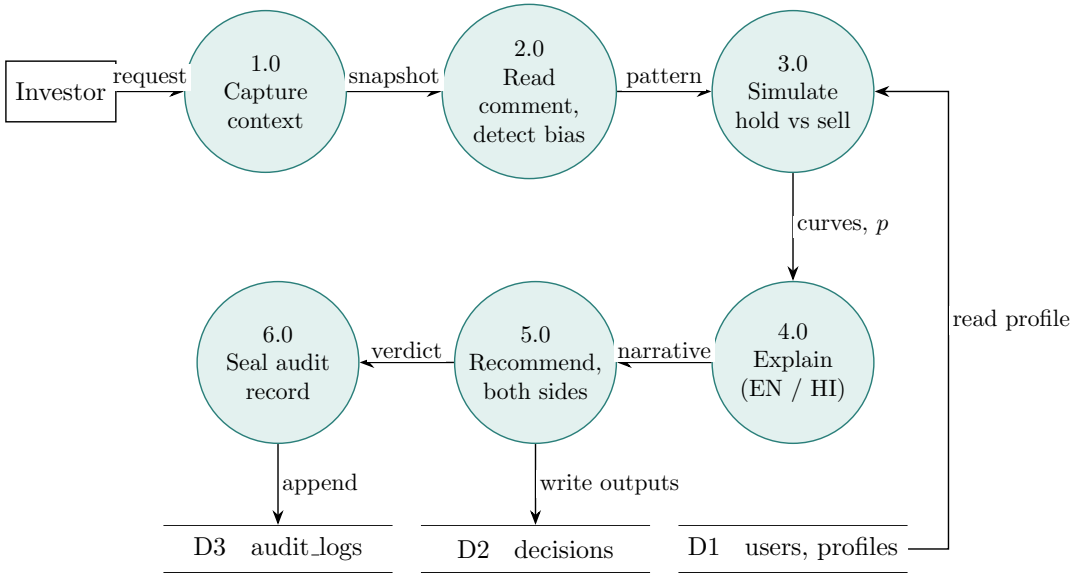


Figure 4.4: Data-flow diagram, level 1 (the analysis process)

Figure 4.4 breaks the analysis into six processes. Process 1.0 is the analyze endpoint, which captures the context, and processes 2.0 to 6.0 are the five pipeline steps. Each process writes its outputs into the decisions store, and the next process reads the refreshed row, which is why the orchestrator reloads the decision before every step.

The simulation reads the profile for income, SIP and the destination of sale proceeds, and the last process appends an entry to the audit store.

4.4 Use Case Diagram

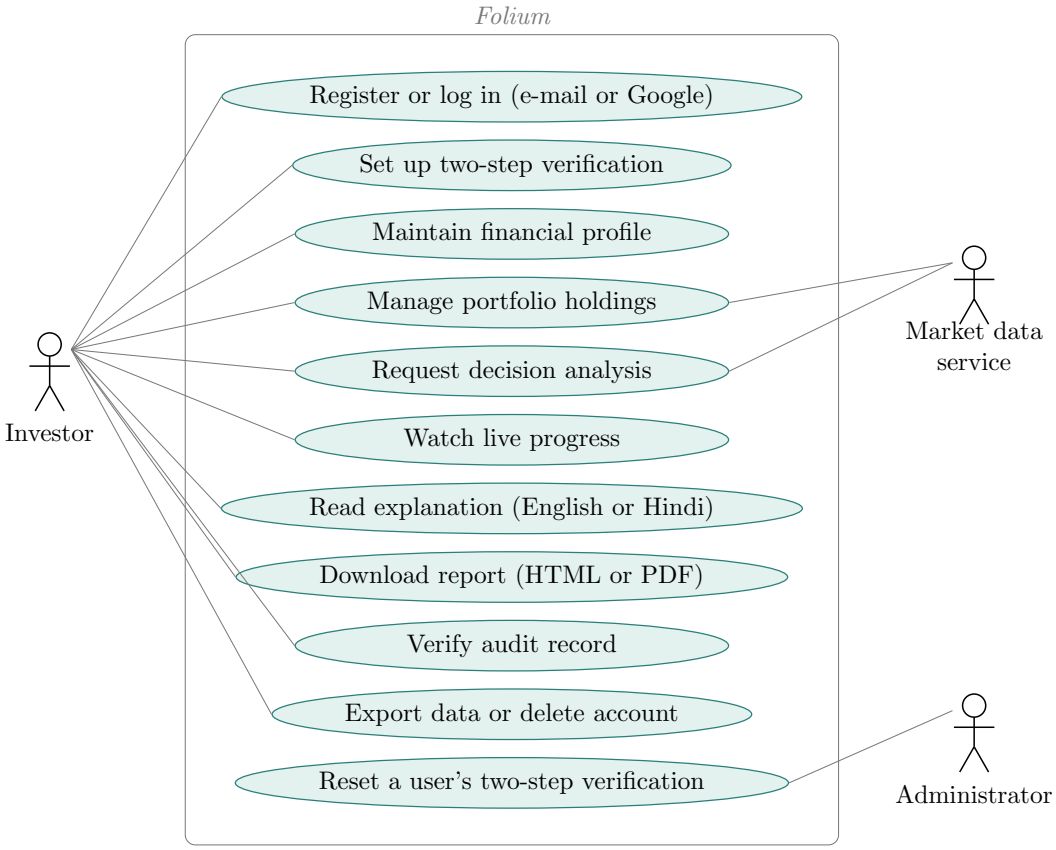


Figure 4.5: Use case diagram

Figure 4.5 shows the actors and their use cases. The investor can register or log in by e-mail or with Google and then uses every feature of the application. The administrator's implemented use case is resetting a user's two-step verification. The market data service takes part in portfolio management, through live prices, and in analysis requests, through the market context and the stock's measured risk.

4.5 Activity Diagram

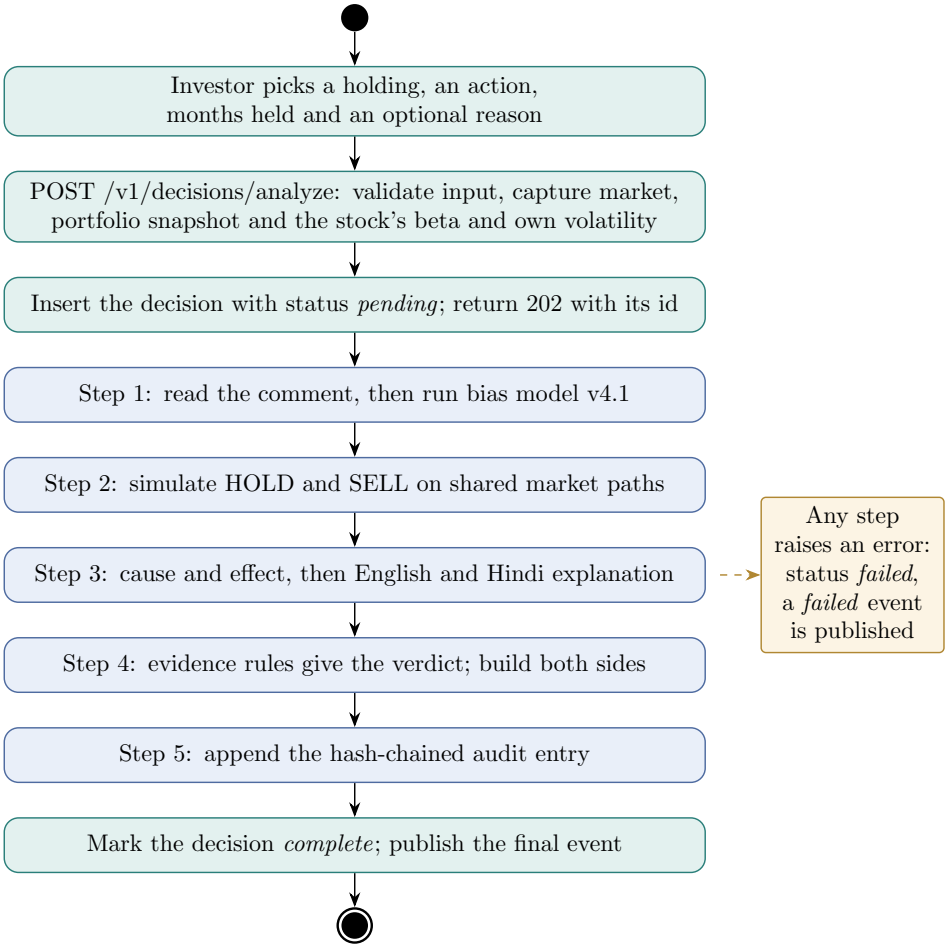


Figure 4.6: Activity diagram of a decision analysis

Figure 4.6 traces one analysis. The synchronous part ends with an HTTP 202 response, so the browser never waits for the models. The five shaded steps run in the background. If any step raises an exception, the decision is marked as failed, the error is stored and a failed event is published so that the browser stops waiting. Problems with wording never fail an analysis, because the explanation step falls back to the template text.

4.6 Sequence Diagram

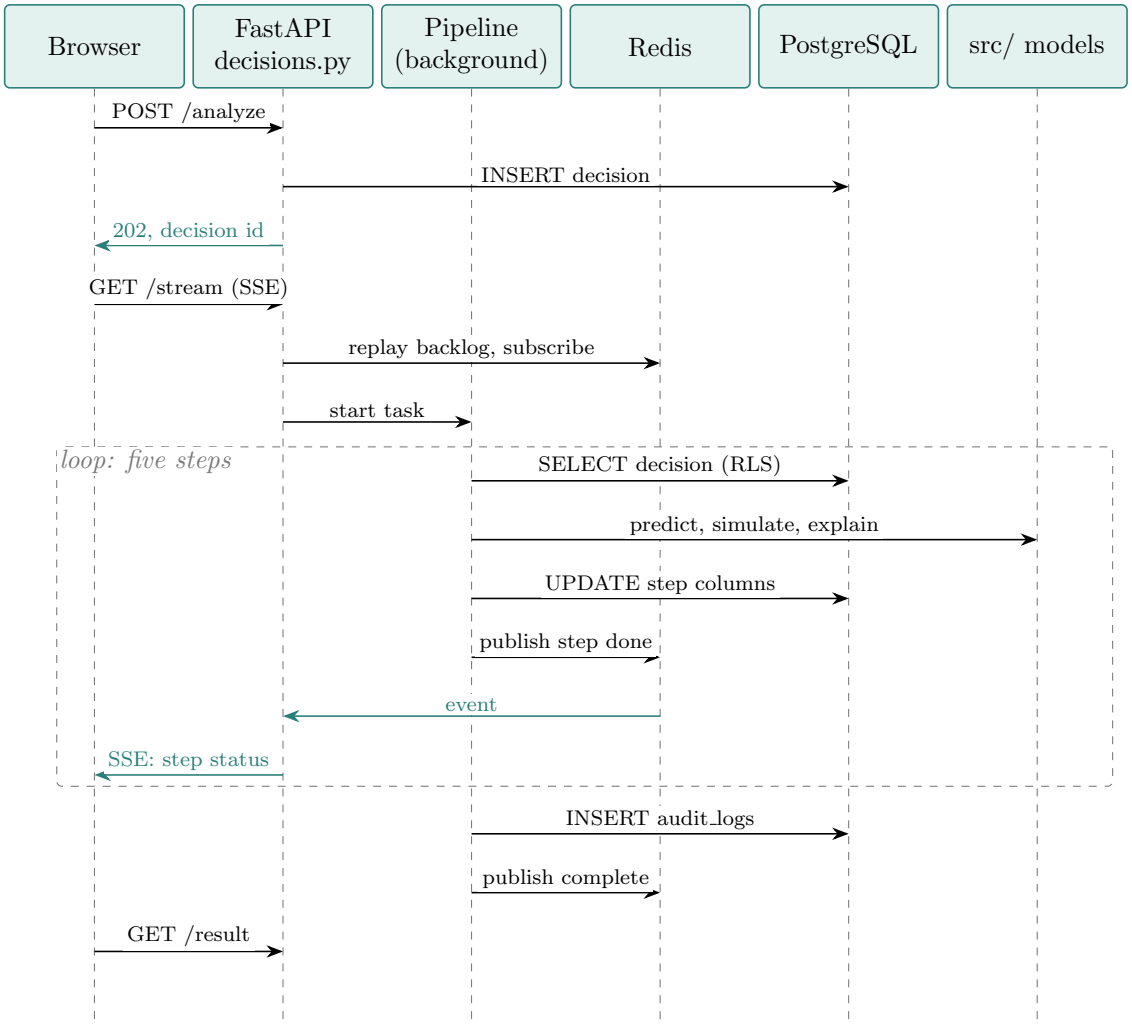


Figure 4.7: Sequence diagram of an analysis with live progress

Figure 4.7 shows the order of messages. One detail is the backlog replay. If the browser only subscribed to Redis publish and subscribe, any event published before the subscription would be lost, and reloading the page during an analysis would show an empty history. Every event is therefore also appended to a Redis list with an expiry time. The stream endpoint first replays this list and then follows live messages, removing duplicates by a sequence number. If the stream cannot be opened at all, the result page falls back to polling.

4.7 Entity-Relationship Diagram

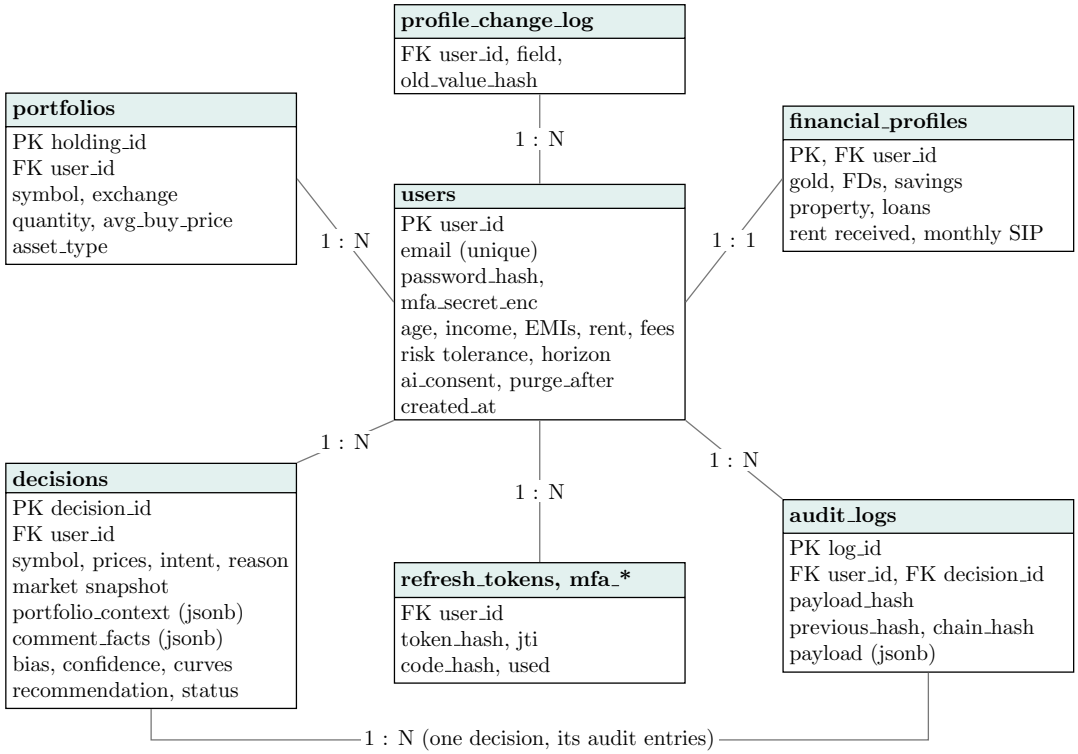


Figure 4.8: Entity-relationship diagram of the main tables

Figure 4.8 shows the main entities, created by Alembic migrations 0001 to 0012. The `users` table holds the account, the basic financial profile, the deletion schedule and the AI consent flag. `financial_profiles` has a one-to-one relationship with users and stores gold, deposits, savings, property, loans, rent received and the monthly SIP, while `portfolios` holds shares and mutual funds. Each analysis is one row of `decisions`, which stores both the inputs (including snapshots of the portfolio and the comment reading as JSONB) and every model output. `audit_logs` holds the hash-chained entries, and its foreign keys use `ON DELETE SET NULL` so that the chain survives when an account is anonymized. Row-level security is enabled and forced on decisions, audit logs, portfolios, financial profiles and the profile change log.

4.8 Deployment Diagram

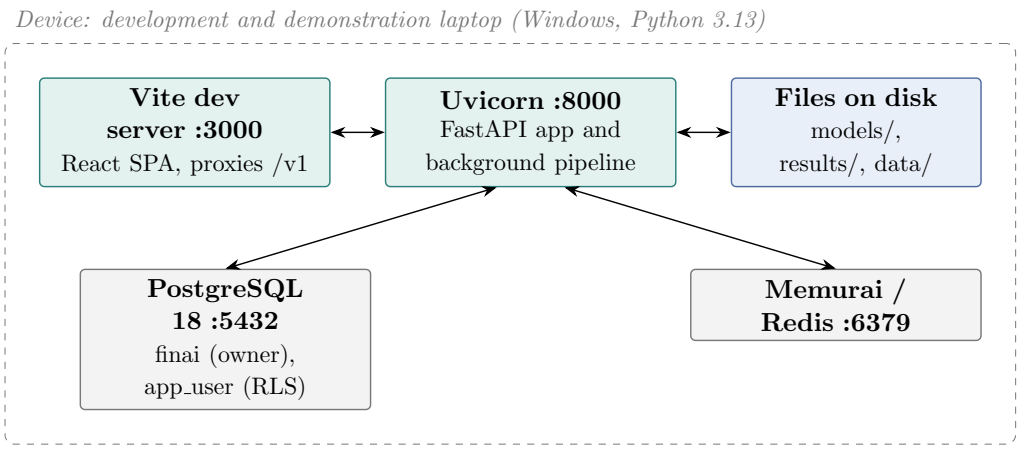


Figure 4.9: Deployment diagram of the development set-up

Figure 4.9 shows the configuration used for development and demonstration, with all processes on one laptop. PostgreSQL has two roles: `finai`, the owner used for migrations, and `app_user`, a non-superuser used by the application so that the row-level security policies apply. Container definitions are present in the repository for the planned public deployment.

4.9 User Interface Design

The public pages are the landing page, the privacy policy and the sign-in pages, together with `/auth/google`, where Google sign-in returns. A first Google sign-in lands on a Welcome page that asks for the basic profile; until it is filled, every other signed-in page except Settings sends the investor there. The application pages are an overview dashboard, the portfolio, a new-analysis form with a live market card, the result page, the history and the settings.

The result page is the centre of the design. It shows the live five-step progress with plain titles such as “Reading your decision” and “Projecting the future”, then the recommendation with its reasons, the position against the whole portfolio, a chart of both futures, the pattern found with probability bars, the six-part explanation with an English and Hindi switch, report downloads, the inputs that weighed most, cause and effect, both sides in numbers, the investor’s own words, “Verify this record” and “How this was worked out”. The interface never shows a model name or an accuracy figure, and confidence is shown in words.

Chapter 5

Methodology

Folium treats an investment decision as the unit of analysis. The investor states what they intend to do with one holding, and the system answers separate questions before combining them: which pattern is most likely behind the decision (the bias model), what each choice would lead to in rupees after tax over the investor's horizon (the simulator), what generally pushes people towards this action and which habit would help (the causal module), and what the investor should do and why (the evidence rules and the explanation engine). Keeping these questions apart is the main methodological choice of the project. Each module reports only what it can support, and the recommendation rules read only the outputs they are designed to read.

The work has an offline part and an online part. Offline, the dataset is built from real market history, the bias model is trained and evaluated, and the simulator and causal module are evaluated; the outputs are model artefacts and result files. Online, for each analysis, the request and its context are captured and the five pipeline steps run. The online path only loads the offline artefacts and never retrains anything.

5.1 Data Sources

Three kinds of data are used. Daily NIFTY 50 closes and India VIX values [51] were downloaded through yfinance [49] and cached (10 June 2008 to 5 October 2026, 4,475 trading days). Monthly NIFTY 50 returns, with a 1.25% dividend yield added, and gold returns in rupees from October 2007 to September 2026 (228 months) are cached for the simulator. Second, the synthetic investor decisions of dataset v4.1 are used for training and causal validation. Third, live share prices, mutual fund NAVs from MFAPI.in [50] and the gold price are fetched at run time and cached for 15 minutes. No real investor's trading records are used.

5.2 Dataset v4.1

There is no public dataset that links the individual decisions of Indian retail investors with their full finances and a ground-truth behavioral label, and building one would require a large consented user study. The decisions are therefore gener-

ated synthetically while the market side is kept real. The design principle is that labels are motives and not rules. Each simulated investor has hidden tendencies, the situation decides which biased motives are possible, these compete with a rational motive, and the winning motive decides both the action and the label. The same sale can therefore be rational for one investor and biased for another, and the model has to infer which from what the app can observe.

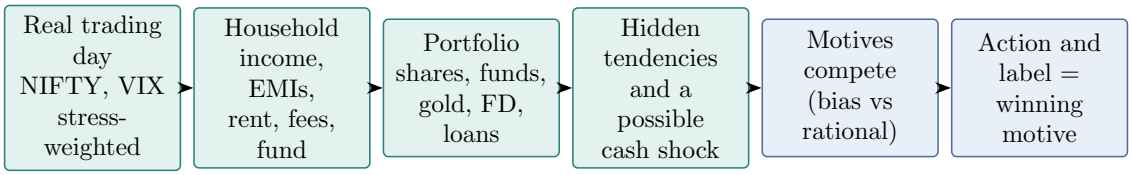


Figure 5.1: Generation of one case in dataset v4.1

Figure 5.1 shows the steps. A real trading day is sampled with stress over-weighted (weights of 1 for normal and bull days, 2 for corrections, 3 for euphoria and 6 for crashes) so that every motive has enough examples, and the day's real one-month return, volatility, VIX and 52-week drawdown are attached. A holding period is drawn from a log-normal distribution with a median of 14 months, and the gain or loss on the position is the real NIFTY move over that period raised to a stock beta, plus stock-specific noise. A household is then generated with age, income, years of investing, risk tolerance, horizon, rent, EMIs, education fees and an emergency fund; 14% of borrowers carry extra debt worth 25 to 65% of income. A portfolio of shares, funds, gold, deposits, savings, property and loans follows. Hidden tendencies are drawn, and a cash shock (job loss, medical bill or fees due) occurs with probability

$$P(\text{shock}) = 0.04 + 0.06\,[\text{no fund}] + 0.12\,\text{clip}\left(\frac{0.30 - s}{0.30}, 0, 1\right), \tag{5.1}$$

where s is the surplus ratio. An emergency fund absorbs 70% of shocks. Investors who genuinely need the money say so with probability 0.65, and others mention it with probability 0.03; this is the observable input `cash_need_stated`.

Label	Motive	Action	Applies when
Loss aversion	panic	sell	at a loss, NIFTY down 2% or more over the month, and VIX $\geq$ 20 or a correction or crash
Loss aversion	refusal	hold	position down 25% or more
Disposition effect	sell a winner	sell	up 8% or more in a normal or rising market
Disposition effect	hold a loser	hold	down 5 to 25% in a normal or rising market
Herd behavior	sell into fear	sell	VIX $\geq$ 22 or NIFTY down 5% or more over the month
Herd behavior	buy into a rally	buy more	NIFTY up 4% or more over the month
Rational	stay the course, add regularly, buy value, average down, cut a loss, rebalance, goal near, needs cash	any	the investor's own situation explains the action

Table 5.1: Label definitions used by the generator

For a case with situation s and investor i, the rational motive competes with weight

$$w_r = 0.7 + 0.025\, y_i + 0.25\, e_i + 3.0\, c_i, \tag{5.2}$$

where y_i is years of investing, e_i indicates an emergency fund and c_i indicates that the money is genuinely needed. A bias motive m with label ℓ competes with weight $w_m = S_\ell\, \tau_{i,\ell}^{1.5}\, I_m(s)$, where $\tau_{i,\ell}$ is the hidden tendency, S_ℓ is 3.4 for loss aversion, 2.0 for herding and 2.8 for disposition, and $I_m(s) \geq 1$ is the intensity of the situation. One motive is sampled in proportion to its weight. The large weight for a genuine cash need encodes the idea that necessity is a constraint and not a preference.

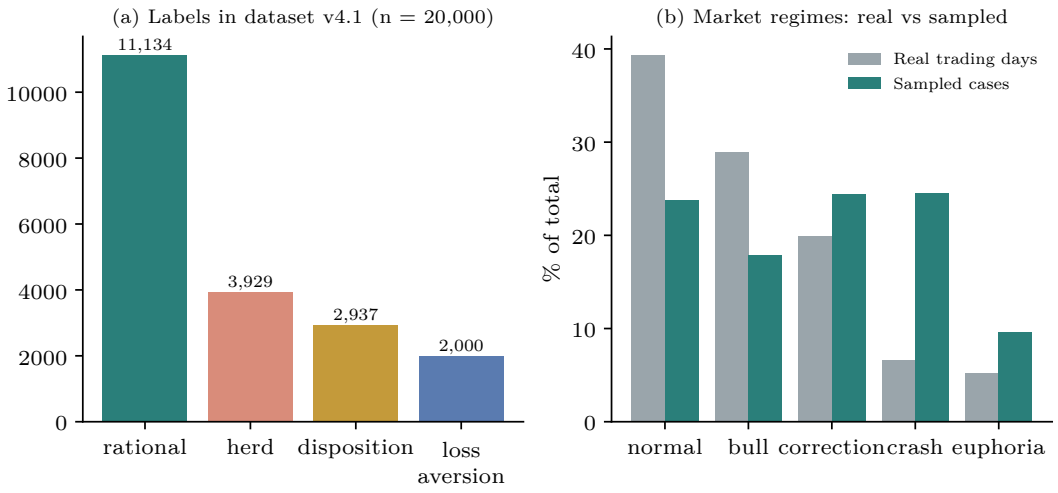


Figure 5.2: (a) Label distribution in dataset v4.1; (b) market regimes among real trading days and sampled cases

The dataset has 20,000 cases (Figure 5.2): 11,134 rational (55.7%), 3,929 herd (19.6%), 2,937 disposition (14.7%) and 2,000 loss aversion (10.0%), with 8,251 holds, 7,995 sales and 3,754 purchases. Crashes form 24.5% of the cases against 6.6% of real days, so every evaluation figure is re-weighted back to real regime frequencies. The median investor is 38 years old with a monthly income of Rs. 75,000 and liquid assets of Rs. 9.7 lakh, and the median position is 12% of liquid assets. Money pressure is represented explicitly: 0.7% of households pay out more than they earn, 44% have no emergency fund, cash shocks hit 6.8% of decisions, and the money is needed in 1,165 cases, 889 of which end in a forced sale.

5.3 Feature Engineering

The same function converts raw inputs into the 22 model features for training and for live requests, so the two cannot drift apart. Besides three action flags, the gain or loss, holding period, market values, age, horizon, risk tolerance and portfolio shares, two ratios are derived:

$$\text{surplus ratio} = \text{clip}\left(\frac{\text{income} + \text{rent received} - \text{fixed outflow}}{\text{income}}, -2, 2\right), \tag{5.3}$$

$$\text{debt ratio} = \text{clip}\left(\frac{\text{loans}}{\text{liquid assets} + \text{property}}, 0, 5\right). \tag{5.4}$$

Three `allows` flags state which biased labels the definitions permit for this situation and action. In the product, missing profile fields are filled with training medians and missing market data with the median real trading day (NIFTY up 1.0% in a month, VIX 16.9); all filled values are listed on the result page.

5.4 Behavioral Classification

Let $\mathbf{x} \in \mathbb{R}^{22}$ be the feature vector and $y \in \{1, \ldots, K\}$ with $K = 4$. The boosted-tree score for class k and its probability are

$$F_k(\mathbf{x}) = \sum_{m=1}^{M} f_{m,k}(\mathbf{x}), \qquad \hat{p}_k(\mathbf{x}) = \frac{e^{F_k(\mathbf{x})}}{\sum_{j=1}^{K} e^{F_j(\mathbf{x})}}, \tag{5.5}$$

and the training objective combines the weighted log-loss with a penalty on tree complexity [5, 26]:

$$\mathcal{L} = \sum_{i=1}^{n} w_i \, \ell\big(y_i, \hat{\mathbf{p}}(\mathbf{x}_i)\big) + \sum_{m,k} \Omega(f_{m,k}), \qquad \Omega(f) = \gamma T + \tfrac{1}{2}\lambda \|\boldsymbol{\omega}\|^2, \tag{5.6}$$

where T is the number of leaves, $\boldsymbol{\omega}$ the leaf weights and w_i the ratio of the real to the sampled frequency of the case's regime.

Guard. Let $m_k(\mathbf{x}) = 1$ if label k is allowed by the definitions, with the rational label always allowed. The guarded probabilities are $\tilde{p}_k = m_k \hat{p}_k / \sum_j m_j \hat{p}_j$, so the model can never output a pattern the definitions rule out.

Calibration. Let c be the probability of the chosen class. A centered isotonic regression [47] fitted on out-of-fold predictions maps c to a calibrated value; unlike plain isotonic regression, which is a step function, it joins the weighted centers of the constant blocks with straight lines. The calibrated value is floored so that the chosen class can never fall below the runner-up, and is capped at 0.99. Calibration is measured by the expected calibration error over ten confidence bins:

$$\mathrm{ECE} = \sum_{b=1}^{10} \frac{W(B_b)}{W} \left|\mathrm{acc}(B_b) - \mathrm{conf}(B_b)\right|. \tag{5.7}$$

The confidence is shown to the investor as High (0.85 and above), Medium (0.65 and above) or Low.

Attribution. For the chosen class, SHAP decomposes the score additively, $F_k(\mathbf{x}) = \phi_{0,k} + \sum_j \phi_{j,k}(\mathbf{x})$ [7], and exact TreeSHAP values [35] are summed into 18 plain drivers such as "gain or loss" or "market fear". Because Shapley values are additive this grouping is exact; the largest additivity error on the evaluation checks is 8.1×10^{-6}.

5.5 Paired Hold versus Sell Simulation

Let r_t^E and r_t^G be the monthly log-returns of NIFTY 50 and of gold in rupees. To keep the swings of history but set typical growth to stated long-run rates ($\mu_E = 11\%$, $\mu_G = 9\%$), each series is re-centered,

$$\tilde{r}_t^E = r_t^E - \bar{r}^E + \frac{\ln(1+\mu_E)}{12}, \qquad \tilde{r}_t^G = r_t^G - \bar{r}^G + \frac{\ln(1+\mu_G)}{12}. \tag{5.8}$$

A path of $12H$ months is built from 12-month blocks whose starting months are drawn uniformly, with indices taken modulo the history length (a circular block

bootstrap [12, 33]). The same indices are used for equity and gold, which keeps their real month-by-month relationship. Re-centering is needed because gold compounded at about 14.8% a year from 2007 to 2026, and replaying that as the future would build an unusually strong gold boom into every path.

For a stock with beta β and company-specific monthly volatility σ, the typical growth follows the capital asset pricing relation with the fixed deposit rate $r_f = 6.75\%$, $\mu_{\text{pos}} = r_f + \beta(\mu_E - r_f)$, and the monthly log-return on path s is

$$\rho_{s,t} = \frac{\ln(1+\mu_{\text{pos}})}{12} + \beta\big(\tilde{r}^E_{s,t} - \bar{\tilde{r}}^E\big) + \varepsilon_{s,t}, \qquad \varepsilon_{s,t} \sim \mathcal{N}\Big(-\tfrac{\sigma^2}{2}, \sigma^2\Big). \tag{5.9}$$

With V_0 the value today, C the cost basis, $\kappa = 0.15\%$ the sale charges and m_0 the months already held, the value in hand after holding for t months is

$$W^{(H)}_{s,t} = V_t(1-\kappa) - \text{Tax}_E\big(V_t - C,\ m_0 + t\big), \qquad V_t = V_0\, e^{\sum_{u \le t} \rho_{s,u}}. \tag{5.10}$$

Selling realizes $P = V_0 - \text{Tax}_E(V_0 - C, m_0) - \kappa V_0$ today and moves it into the investor's existing mix of gold, deposits and savings, with shares $\alpha_G, \alpha_F, \alpha_S$:

$$W^{(S)}_{s,t} = \alpha_G P\, G_{s,t} - \text{Tax}_G(\cdot) + \alpha_F P\big(1 + r_f(1-\tau)\big)^{t/12} + \alpha_S P\big(1 + r_{sv}(1-\tau)\big)^{t/12}, \tag{5.11}$$

where τ is the investor's marginal slab rate on interest and $r_{sv} = 3\%$. Because both values are computed on the same path, their difference isolates the decision. The outputs are

$$\tilde{\Delta}_T = \text{median}_s\big(W^{(H)}_{s,T} - W^{(S)}_{s,T}\big), \tag{5.12}$$

$$p = \frac{1}{n}\sum_{s=1}^{n} \mathbf{1}\big[W^{(H)}_{s,T} > W^{(S)}_{s,T}\big], \tag{5.13}$$

together with the 10th, 50th and 90th percentiles of both paths for every year. The median is used because single-stock outcomes are right-skewed, so the mean would overstate the usual result.

Item	**Rule (FY 2025-26, new regime)**
Listed equity held 12 months or less	20% short-term gains tax
Listed equity held longer	12.5% on gains above Rs. 1,25,000
Gold	12.5% after 24 months; otherwise the slab rate
FD and savings interest	The investor's marginal slab rate, every year
Cess	4% on top of each tax
Slab rate	Taxable income is gross minus Rs. 75,000; nil up to Rs. 12 lakh taxable (Section 87A rebate); above that slabs of 0, 5, 10, 15, 20, 25 and 30% in steps of Rs. 4 lakh

Table 5.2: Tax rules used by the simulator

The tax rules in Table 5.2 follow the Finance (No. 2) Act, 2024 [52] and the Finance Act, 2025 [53]. The slab rate depends on income before tax, but the app records

take-home pay. If $T(g)$ is the income tax on gross income g, including the rebate and its marginal relief, then take-home pay is $h(g) = g - T(g)$, and the simulator needs $\hat{g}$ such that $h(\hat{g})$ equals twelve times the monthly take-home pay. Since h is increasing except in a short stretch of the marginal relief band, $\hat{g}$ is found by bisection with 60 iterations, and rent received is added at 70%. For example, a take-home pay of Rs. 1.6 lakh a month corresponds to about Rs. 21.5 lakh gross, so interest is taxed at 26% and not the 20.8% that the take-home figure alone would give.

5.6 Causal Estimation

For a treatment A and an outcome Y (whether the investor takes a given action), the average treatment effect and its identification by the g-formula under consistency, positivity and conditional exchangeability given an adjustment set Z are [10, 36, 37]

$$\mathrm{ATE} = \mathbb{E}[Y \mid \mathrm{do}(A = a')] - \mathbb{E}[Y \mid \mathrm{do}(A = a)], \tag{5.14}$$

$$\mathbb{E}[Y \mid \mathrm{do}(A = a)] = \sum_{z} \mathbb{E}[Y \mid A = a, Z = z]\, P(Z = z). \tag{5.15}$$

Two estimators work from the data alone, as a real study would. G-computation fits a histogram gradient-boosting model of the action given everything observed (scikit-learn [48], 400 iterations) and averages its predictions with the treatment set to each value. DoWhy [11] identifies the effect from a graph that mirrors the generator and estimates it with a linear model, followed by placebo, random common cause and subset refutation tests.

Because the generator is known exactly, the true effect can also be computed by re-evaluating the generator's motive probabilities with the input changed. This is what makes the evaluation possible: the estimates can be compared with the truth, which is never available in a real observational study. Gold share has no causal path to the decision and is used as a negative control. A habit is modeled as halving the tendency it acts on, which is an assumption stated in the app.

5.7 Evidence-Based Recommendation

Intent	**Rule**
Sell	$p \geq 0.60$ gives HOLD; $p \leq 0.40$ gives SELL; otherwise HOLD if the sale reads as biased, else SELL
Hold	$p \leq 0.40$ gives SELL; otherwise HOLD
Buy more	Herd behavior, or any bias with confidence ≥ 0.60, gives HOLD; otherwise BUY MORE if $p \geq 0.50$, else HOLD

Table 5.3: Recommendation rules

The verdict comes from the small rule set in Table 5.3, where p is the share of simulated futures in which holding beats selling. The band between 0.40 and 0.60 is treated as a toss-up, in which the behavioral reading decides. A verdict about money should be checkable by hand, and a rule set with stated thresholds meets

this need where a language model would not. Every verdict stores its reasons and the exact numbers used.

5.8 Explanation

All model outputs are gathered into one dictionary of facts. Hand-written Jinja2 [54] templates, identical in structure in English and Hindi, turn the facts into six sections: in short, what is going on right now, your two possible futures, the story of your money, why we are telling you this, and what to do now. An optional Gemini rewording is accepted only if it introduces no new number or rupee amount and its length stays between 0.6 and 1.8 times the original.

5.9 Tamper-Evident Audit Chain

Let canon$(\cdot)$ be a canonical JSON serialization (sorted keys, no spaces) and H be SHA-256. For the i-th entry of an investor, with payload P_i,

$$h_0 = H(\texttt{GENESIS:} \| \text{user id} \| : \| \text{created at}), \tag{5.16}$$

$$\pi_i = H(\text{canon}(P_i)), \qquad h_i = H(h_{i-1} \| \pi_i). \tag{5.17}$$

Changing any byte of a payload changes π_i and therefore every later chain hash, and removing an entry leaves the next one pointing at a hash that no entry produces, following the linked time-stamping idea of Haber and Stornetta [44]. Verification walks the entries by following the links from the genesis value rather than by timestamp, and reports an entry as altered if its hashes do not recompute, as unlinked if it does not connect to the chain, and as a fork if two entries claim the same predecessor.

Chapter 6

Implementation

This chapter describes the implementation module by module. The code listings are excerpts copied from the repository; omitted lines are marked with ..., and identifiers and logic are unchanged.

Path	Contents
`backend/`	Product API: routes, authentication, the analysis pipeline and its steps, services, database layer and migrations, middleware and tests
`frontend/`	React application: pages, components, API client, query hooks and tests
`src/`	Models and engines: dataset generator, bias model, simulator, causal module, explainer and templates; earlier versions kept for rollback
`models/`, `data/`	Trained model files; the dataset and cached market history
`results/`, `scripts/`	Evaluation files and reports; training, evaluation and maintenance scripts
`tests/`, `notebooks/`	Tests for the models and engines; research notebooks

Table 6.1: Organization of the repository

6.1 Dataset Generator

The generator in `src/dataset_v4.py` produces the 20,000 labelled decisions. Its most important function for the rest of the system is `applicable_bias_motives`, which encodes the label table, together with `allowed_labels`, which the bias model uses both as an input and as a guard. Keeping the table in one place means that the generator, the model and the tests cannot disagree about what a label means.

Listing 6.1: The label rules in code form (`src/dataset_v4.py`)

```
def applicable_bias_motives(s: dict) -> list[Motive]:
    gl, r1m, vix, regime = s["unrealized_gl"], s["nifty_1m_return"], s["vix"], s
    ["regime"]
```

```
    falling_fear = gl < -0.03 and r1m <= FALLING_1M and (vix >= FEAR_VIX or
    regime in ("crash", "correction"))
    calm = regime in ("normal", "bull")
    out: list[Motive] = []
    if falling_fear:
        out.append(Motive("loss_aversion_panic", "loss_aversion", "sell"))
    if gl <= DEEP_LOSS:
        out.append(Motive("loss_aversion_refusal", "loss_aversion", "hold"))
    if calm and gl >= EARLY_WIN:
        out.append(Motive("disposition_sell_winner", "disposition", "sell"))
    if calm and DEEP_LOSS < gl <= MODERATE_LOSS:
        out.append(Motive("disposition_hold_loser", "disposition", "hold"))
    if vix >= HERD_FEAR_VIX or r1m <= HERD_FEAR_1M:
        out.append(Motive("herd_sell_fear", "herd", "sell"))
    if r1m >= FRENZY_1M:
        out.append(Motive("herd_buy_frenzy", "herd", "buy_more"))
    return out
```

The thresholds are module constants, so the dataset card, the tests and the code all refer to the same values. Using `numpy.random.default_rng(2026)` with the cached market file makes the dataset identical on every rebuild.

6.2 Bias Model

`src/bias_model_v4.py` builds the features, loads the model and makes predictions. The function `build_features` refuses to run with missing inputs or an unknown action, so that the caller fills any gap and can disclose what it filled. The model is stored as native XGBoost JSON with a metadata file that holds the feature list, the calibration curve and the training medians, and `load` refuses to run if the stored feature list differs from the code.

Listing 6.2: The definitional guard and top-label calibration (`src/bias_model_v4.py`)

```
def apply_guard(p: np.ndarray, mask: np.ndarray) -> np.ndarray:
    """Zero the patterns the definitions rule out, then renormalise."""
    q = p * mask
    return q / q.sum(axis=1, keepdims=True)

def calibrate_top_label(p, curve_x, curve_y):
    p = np.asarray(p, dtype=float)
    k = p.argmax(axis=1)
    rows = np.arange(len(p))
    top = p[rows, k]
    rest = 1.0 - top
    second = np.sort(p, axis=1)[:, -2]
    c = np.interp(top, curve_x, curve_y)
    floor = np.where(rest > 0, second / np.maximum(rest + second, 1e-12) + 1e-6,
     top)
    c = np.clip(np.maximum(c, floor), 0.0, 1.0)
    scale = np.where(rest > 1e-12, (1.0 - c) / np.maximum(rest, 1e-12), 0.0)
    q = p * scale[:, None]
    q[rows, k] = c
    return q
```

Since the rational label is always allowed, the guard never removes every class.

`calibrate_top_label` replaces only the chosen class's probability and rescales the others in their original ratios. The `floor` term ensures that calibration can lower the stated confidence but cannot demote the chosen class below the runner-up; it changes how sure the model says it is, never what it says. The `predict` function asks XGBoost for both the raw scores and the exact SHAP contributions (`pred_contribs=True`), and only the contributions of the chosen class are summed into drivers. This fixed a defect of the earlier v3 model, whose "what weighed most" explained a different class from the one it displayed in 98.6% of test cases.

Training is done by `scripts/train_bias_v4.py`. The cases are split 80:20 into development and test sets, stratified by label and regime. Five-fold cross-validation with early stopping chooses the number of trees, and the same folds give out-of-fold predictions for fitting the calibration curve. The hyperparameters are listed in Table 7.1.

In the product, `backend/core/bias_detector.py` assembles the inputs from the decision row, the profile and the portfolio snapshot taken at submission. The comment reader runs first so that `cash_need_stated` is available. It scans the reason with regular expressions in English, Hinglish and Hindi, for example `(paisa|paise)` `(chahiye|ki zarurat)` or `(lost my job|laid off)`, and returns each match with the whole sentence quoted verbatim. Only a stated fact (a need for the money) changes a model input. Feelings such as fear or following friends are quoted back to the investor but never change the reading or the verdict. When the investor has consented, Gemini may also read the comment, but its answer is accepted only if every quote appears word for word in the comment.

6.3 Simulator

`src/simulator_v2.py` compares holding and selling one position. Its input is a dataclass with the position value, cost basis, months held, asset type, horizon (clamped to 1 to 30 years), income, the rest of the liquid portfolio, beta and own volatility, SIP and education fees, the number of paths and the seed.

Listing 6.3: Circular 12-month block bootstrap (`src/simulator_v2.py`)

```
def _bootstrap(rng: np.random.Generator, n_paths: int, months: int, n_hist: int)
     -> np.ndarray:
    b = int(ASSUMPTIONS["block_months"])
    n_blocks = -(-months // b)
    starts = rng.integers(0, n_hist, size=(n_paths, n_blocks))
    return ((starts[:, :, None] + np.arange(b)) % n_hist).reshape(n_paths, -1)
    [:, :months]
```

The function in Listing 6.3 is fully vectorized. It draws all block starts at once, expands each into 12 consecutive month indices, wraps them with the modulo operator and trims the result to the required length. Without the wrap-around, the first and last months of history would be under-sampled, and October 2007 starts with the 2008 crash.

Listing 6.4: Paired HOLD and SELL valuation after tax (excerpt of `simulate`)

```
    growth = a["fd_rate"] + beta * (market_growth - a["fd_rate"])
```

```
    eq_shock = eq_log[idx] - eq_log.mean()
    own = rng.normal(-own_vol ** 2 / 2, own_vol, size=(n, m))
    pos_cum = np.exp(np.cumsum(math.log1p(growth) / 12 + beta * eq_shock + own,
    axis=1))
    ...
    hold_value = inp.position_value * pos_cum
    hold_gain = hold_value - inp.cost_basis
    hold_in_hand = hold_value * (1 - a["sale_cost"]) - equity_gains_tax(
    hold_gain, inp.months_held + months_ahead)
    sale = sale_today(inp)
    split = destination(inp)
    p = float(sale["proceeds"])
    gold_part = p * split["gold"] * gd_cum
    gold_in_hand = gold_part - gold_gains_tax(gold_part - p * split["gold"],
    months_ahead, slab)
    sell_in_hand = (gold_in_hand + p * split["fixed_deposits"] * fd_after_tax
                    + p * split["savings"] * sav_after_tax)
    ...
    diff = hold_end - sell_end
    opp = float(np.median(diff))
    prob_hold = float((diff > 0).mean())
```

Listing 6.4 shows three design points. The holding's growth uses the capital asset pricing relation, so a volatile stock is not projected to grow faster only because it is volatile. The company-specific shock has mean $-\sigma^2/2$ in log space, so its expected multiplicative effect is one. HOLD is valued as if cashed in each year, while SELL paid its tax on day one and is taxed on interest every year, so the two are compared like for like. The tax helpers work on whole arrays, so tax is computed for all paths and months in one call. In the product, `backend/core/simulator.py` builds the inputs from the decision and the profile. It uses the beta and own volatility measured at submission from ten years of monthly prices, falling back to the median NIFTY 50 stock (beta 1.0, own volatility 21%), and it converts take-home pay to gross income with `gross_from_take_home` before looking up the slab rate.

6.4 Causal Module

`src/causal_v4.py` re-implements the generator's motive weights so that, for any case, it returns the exact probability of selling, holding and buying more. It matches the stored motive share of all 20,000 cases with a maximum absolute error of zero. The true effect of an intervention is the mean change in these exact probabilities. The estimated effect comes from the g-computation model of Listing 6.5 and from DoWhy with three refutation tests. The back end reads the results file and caches it.

Listing 6.5: G-computation estimator (`src/causal_v4.py`)

```
class GComputation:
    """S-learner: P(action | everything observed), queried under interventions.
    """
    def __init__(self, df: pd.DataFrame, seed: int = 0):
        from sklearn.ensemble import HistGradientBoostingClassifier
        self.model = HistGradientBoostingClassifier(max_iter=400, learning_rate
    =0.06, max_leaf_nodes=31,
```

```
                                                    l2_regularization=1.0,
random_state=seed)
        self.model.fit(df[OBSERVED], df.action.map({a: i for i, a in enumerate(
ACTIONS)}))

    def probabilities(self, df: pd.DataFrame) -> np.ndarray:
        p = self.model.predict_proba(_refresh_derived(df)[OBSERVED])
        out = np.zeros((len(df), 3))
        out[:, self.model.classes_] = p
        return out
```

6.5 Recommendation and Both Sides

The function `decide` in `backend/core/recommendation.py` is a pure function. It takes values and returns a verdict and a list of reasons without touching the database or the network, which makes it directly testable.

Listing 6.6: Evidence rules for the sell intent (excerpt of `decide`)

```
    if intent == "sell":
        if p >= _STRONG:
            reasons.append(f"Holding beat selling in {pct} of {horizon_years}-
year simulations.")
            if biased:
                reasons.append(f"The urge to sell shows signs of {bias_name}.")
            return "HOLD", reasons
        if p <= _WEAK:
            reasons.append(f"Holding beat selling in only {pct} of simulations."
)
            return "SELL", reasons
        reasons.append(f"The simulations are close to even ({pct} favour holding
).")
        if biased:
            reasons.append(f"With {bias_name} detected, the cautious choice is
to wait.")
            return "HOLD", reasons
        reasons.append("No bias pattern showed up behind the sale, so your call
stands.")
        return "SELL", reasons
```

If no simulation result is available, the odds are treated as even and that fact is stated. When a reason mentions the confidence of a reading, it is written in words (high, medium or low) and never as a percentage. After the verdict, `backend/core/debate.py` builds the case for each side from the analysis's own numbers. These cases are never read by the verdict.

6.6 Pipeline Orchestrator

Listing 6.7: The step loop and failure handling (excerpt of `run_analysis_pipeline`)

```
    try:
        for step_name in STEP_ORDER:
            await _set_status(db, decision_id, user_id, "processing", step_name)
```

```
            await publish_event(redis, decision_id, {"step": step_name, "status"
    : "running"})
            decision = await _load_decision(db, decision_id, user_id)
            ctx = PipelineContext(decision_id=decision_id, user_id=user_id,
                                  decision=decision, db=db, redis=redis)
            result = await resolve_step(step_name)(ctx)
            await _apply_updates(db, decision_id, user_id, result.updates)
            await publish_event(redis, decision_id,
                                {"step": step_name, "status": "done", **result.
    event})
        await _set_status(db, decision_id, user_id, "complete", None, completed=
    True)
        await publish_event(redis, decision_id, {"step": "complete", "status": "
    done"})
    except Exception as exc:
        sentry_sdk.capture_exception(exc)
        try:
            await _set_status(db, decision_id, user_id, "failed", None,
    error_message=str(exc)[:500])
        finally:
            await publish_event(redis, decision_id,
                {"step": "failed", "status": "error", "error": str(exc)[:500]})
```

The loop in Listing 6.7 guarantees two properties. Each step sees what the earlier steps wrote, because the row is reloaded every time. A connected browser always learns that the run has ended: the success path publishes *complete*, and the failure path publishes *failed* inside a `finally` block, so the stream is notified even if writing the status fails.

6.7 Explanation Engine

The explanation engine in `src/explainer.py` gathers all numbers into one facts dictionary and renders the six sections with Jinja2 templates. Templates are rendered with `StrictUndefined`, so a missing fact raises an error in testing instead of printing a blank. Internal terms are translated into plain phrases, a 30-term jargon blocklist is enforced by tests, and money is formatted the Indian way (Rs. 1.2 lakh in prose, Rs. 1,32,451 in tables). The English and Hindi banks have 34 templates each with identical structure, so Hindi works without any external service. Reports are produced as HTML in both languages and as PDF in English; a Hindi PDF is refused with an explanation because ReportLab cannot shape Devanagari, and the HTML report prints correctly from a browser.

Listing 6.8: Number-preservation check for optional AI wording (`src/llm_polish.py`)

```
def guard_ok(original: str, polished: str) -> bool:
    if not polished or not polished.strip():
        return False
    invented_numbers = _numbers(polished) - _numbers(original)
    if invented_numbers:
        return False
    invented_money = _money(polished) - _money(original)
    if invented_money:
        return False
    ratio = len(polished) / max(len(original), 1)
```

```
    if not 0.6 <= ratio <= 1.8:
        return False
    return True
```

Tables are replaced with placeholder tokens before any text is sent and restored afterwards. Any failure (no key, quota, timeout or a malformed reply) returns the template text, and two consecutive failures pause the feature for five minutes. The check in Listing 6.8 rejects added numbers; a removed number is caught only through the length limit, which is noted as an open item in Chapter 8.

6.8 Data Layer and Row-Level Security

Listing 6.9: Per-transaction user for row-level security (`backend/db/pool.py` and migration 0007)

```
    async def rls_transaction(self, user_id):
        async with self._pool.acquire() as conn:
            async with conn.transaction():
                await conn.execute(
                    "SELECT set_config('app.current_user_id', $1::text, true)",
                    str(user_id) if user_id is not None else "")
                yield conn

# policy created for each protected table
CREATE POLICY decisions_isolation ON decisions FOR ALL
    USING (user_id = NULLIF(current_setting('app.current_user_id', true), '')::
    uuid)
    WITH CHECK (user_id = NULLIF(current_setting('app.current_user_id', true), '
    ')::uuid);
```

The third argument of `set_config` makes the setting local to the transaction, so it cannot leak to the next request that reuses a pooled connection, and `NULLIF` turns an unset value into `NULL`, so a query that forgot to set the user sees no rows instead of failing. PostgreSQL superusers bypass row-level security, so the application connects as the non-superuser `app_user`. Another user's decision is simply invisible, and a request for it returns 404 rather than 403.

6.9 Audit Chain

Listing 6.10: Appending an audit entry under a per-investor lock (`backend/core/audit.py`)

```
async def append_entry(conn, user_id, decision_id, row, recorded_at=None):
    await conn.execute("SELECT pg_advisory_xact_lock(hashtextextended($1::text,
    0))", str(user_id))
    entries, genesis = await load_chain(conn, user_id)
    previous = chain_head(entries, genesis)
    payload = build_payload(row, recorded_at or datetime.now(timezone.utc))
    payload_hash = sha256(canonical(payload))
    chain_hash = sha256(previous + payload_hash)
    await conn.execute(
        "INSERT INTO audit_logs (decision_id, user_id, event_type, payload_hash,
     "
```

```
        "previous_hash, chain_hash, payload) VALUES ($1, $2, $3, $4, $5, $6, $7
    ::jsonb)",
        decision_id, user_id, payload["event"], payload_hash, previous,
    chain_hash, canonical(payload))
```

The payload holds two fingerprints: one over 21 input fields (the holding, prices, action, reason, market and portfolio snapshots and the comment reading) and one over 14 output fields (pattern, confidence, projections, explanation, transcript and verdict). Values are normalized so that a number reads the same however it arrives from the database. The advisory lock serializes appends for each investor, so two analyses finishing together cannot claim the same predecessor. At the database level, rules turn `UPDATE` and `DELETE` on `audit_logs` into no-ops, so the table is append-only.

6.10 Authentication and Google Sign-in

Concern	Implementation
Passwords	bcrypt with cost 12; zxcvbn score of 3 or more required
Sessions	RS256 JWT, 15-minute access and 7-day refresh tokens with rotation, in httpOnly `SameSite=Lax` cookies; a Redis blacklist invalidates tokens on logout and refresh
CSRF	The `csrf_token` cookie must equal the `X-CSRF-Token` header on every mutating request
Two-step	TOTP with `valid_window=1`; secret encrypted with Fernet; eight bcrypt-hashed backup codes; recovery requests limited to one per e-mail per day
Google sign-in	OpenID Connect through Authlib; single-use state in Redis for ten minutes; only addresses verified by Google are accepted
Rate limits	Login 5 per minute, registration and password reset 3 per hour, analysis 10 per minute

Table 6.2: Authentication and session controls

For Google sign-in, `GET /v1/auth/google/login` creates a random single-use state and stores it in Redis with the "keep me signed in" choice and the page to return to (only paths inside the app are accepted). When Google sends the browser back, the callback consumes the state, exchanges the code for the profile and accepts it only if Google reports the e-mail as verified; otherwise anyone could link an unverified address to another person's account. The callback always redirects to `/auth/google` in the app with the outcome in the URL fragment. Browsers never send a fragment to a server, so the single-use tokens it may carry stay out of server logs, and the page removes the fragment from the address bar once it has read it. The page then signs the investor in, asks for the two-step code, offers "Keep my account" during the deletion grace period, or explains a failure. A new Google account has no financial profile yet, so the route guard sends it to the Welcome page until the profile is complete, and a Google-only account can set a password from Settings.

6.11 Account Deletion and Data Export

Deleting an account needs the password, and a TOTP code if two-step verification is on, and signs the user out everywhere. For 30 days, logging in offers "Keep my account". After that, a periodic task anonymizes the account: name, e-mail, credentials, profile and financial data, holdings, sessions and profile history are erased, and each analysis loses its comment, snapshots, explanation and projections. The user identifier and creation time, which anchor the audit chain, remain with the audit entries but are no longer linked to any person. "Download my data" returns everything held about the investor as JSON, without the password hash, the two-step secret or tokens.

6.12 Front End

The front end is a React 19 application written in TypeScript and built with Vite. An axios client sends the CSRF token on mutating requests and, when a session expires, refreshes it once and retries. TanStack Query hooks fetch every resource, forms use React Hook Form with Zod schemas, and charts use Recharts. The result page follows the analysis through an `EventSource` on the stream endpoint and falls back to polling if the stream fails. Route guards send signed-out visitors to the login page while remembering where they were going.

6.13 Error Handling and Edge Cases

Situation	Behavior
No live price	Valued at average cost and marked “price unavailable”
Market data unavailable	A calm median market is assumed and disclosed
Profile or portfolio gaps	Training medians used and listed on the result page
Stock with too little history	Median NIFTY 50 stock (beta 1.0, own swings 21%) used and disclosed
No horizon given	20 years, stated on the page; horizons above 30 are capped
Payments exceed income	Surplus ratio clipped at -2; a stated cash need reads a sale as rational
Very large position or goal within 3 years	A sale can be read as rational
Page reloaded during an analysis	Event backlog replayed; polling fallback
Two analyses finishing together	Per-investor lock keeps the audit chain linear
First sign-in with Google	Account created; profile asked on the Welcome page
Google sign-in cancelled or refused	The app explains what happened and offers to try again
AI help fails or misquotes	Rule-based reading and template text are used

Table 6.3: Edge cases and how they are handled

Chapter 7

Results and Analysis

Every number in this chapter is taken from the evaluation files committed in the `results/` folder of the repository: the bias model evaluation written by the training script, the simulator evaluation, the causal results, and the paper evaluation written by `evaluate_for_paper.py`. The charts were plotted directly from these files. The experiments use fixed seeds and cached market data, so they can be reproduced. The behavioral labels are synthetic and the market history is real; no experiment with real investors has been carried out.

7.1 Experimental Setup

The 20,000 cases of dataset v4.1 were split into 16,000 development and 4,000 test cases, stratified by label and market regime, and the test set was used once. Five-fold cross-validation with early stopping on the development set gave best iterations of 423, 330, 329, 381 and 364, and the final model uses 402 trees (1.1 times the mean). Because stress regimes are over-sampled, every metric is re-weighted to the real frequency of each regime (normal 39.4%, bull 29.0%, correction 19.9%, crash 6.6%, euphoria 5.2%), and these are the figures quoted throughout.

Hyperparameter	**Value**	**Hyperparameter**	**Value**
Objective	multi:softprob	Row subsample	0.85
Number of trees	402	Column subsample	0.85
Maximum depth	5	Minimum child weight	3
Learning rate	0.03	L2 regularization (λ)	2.0
Tree method	hist	Split seed / model seed	7 / 42

Table 7.1: Hyperparameters of bias model v4.1

The evaluation metrics are accuracy, per-class precision, recall and F1, macro-F1 (the mean of the per-class F1 scores, which gives each class equal weight since loss aversion is only 10% of cases), log-loss and the expected calibration error defined in Chapter 5. With weighted counts of true positives TP_k, false positives FP_k and

false negatives FN_k:

$$\text{Precision}_k = \frac{TP_k}{TP_k + FP_k}, \qquad \text{Recall}_k = \frac{TP_k}{TP_k + FN_k}, \tag{7.1}$$

$$F1_k = \frac{2\,\text{Precision}_k\,\text{Recall}_k}{\text{Precision}_k + \text{Recall}_k}. \tag{7.2}$$

7.2 Behavioral Classification Results

Metric	As sampled	Real frequency
Accuracy (test)	86.0%	**87.9%**
Macro-F1 (test)	0.834	**0.840**
Expected calibration error (test)	–	**0.014**
Log-loss (test)	–	0.246
Cross-validation accuracy	84.9%	86.3%
Patterns the definitions rule out	–	0%

Table 7.2: Overall test performance of the bias model

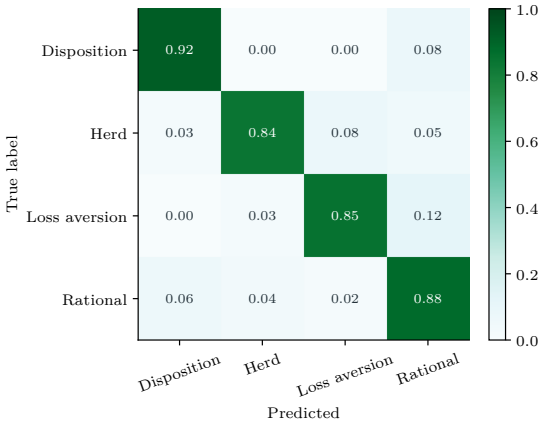


Figure 7.1: Confusion matrix (rows normalized, real frequency)

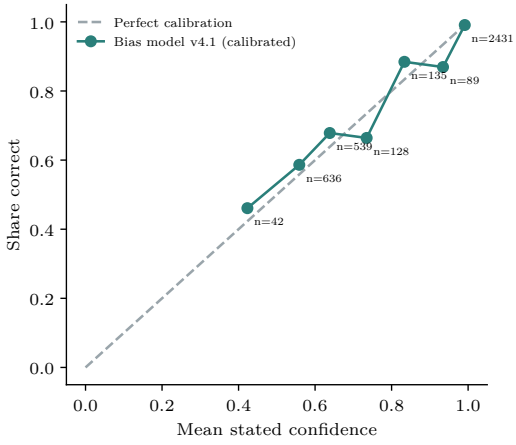


Figure 7.2: Reliability diagram after calibration

Pattern	Precision	Recall	F1
Disposition effect	0.844	0.918	0.879
Herd behavior	0.819	0.838	0.828
Loss aversion	0.667	0.849	0.747
Rational	0.939	0.875	0.906

Table 7.3: Per-class metrics (real frequency)

Table 7.2 gives the overall result: 87.9% accuracy, macro-F1 of 0.840 and an expected calibration error of 0.014 on the test set. The guard and the calibration step did

not change any prediction on the test set. Table 7.3 and Figure 7.1 show that loss aversion is the weakest class. In a falling and fearful market, a loss-making sale is permitted under both loss aversion (panic) and herd behavior (selling into fear), and the data cannot always tell which motive won: 8.0% of true herd cases are read as loss aversion, and 12.2% of true loss aversion cases are read as rational. Rational decisions are recognized with the highest precision (0.939). This matters because wrongly calling a sensible decision biased is the more harmful error for a user.

Band shown to the user	**Threshold**	**Share of analyses**	**Share correct**
High	≥ 0.85	68.4%	**98.7%**
Medium	0.65 to 0.85	7.1%	74.4%
Low	< 0.65	24.5%	61.7%

Table 7.4: Confidence bands and how often each is right

The calibrated points in Figure 7.2 lie close to the diagonal. Most cases (66% by weight) fall in the 0.95 to 1.00 bin, where the model is right 99.1% of the time. The bands in Table 7.4 are what the investor sees: a High reading is right about 99 times in 100, while a Low reading usually means that two patterns are close to even. In 44.3% of cases the definitions allow only one label (holding a winner in a calm market, for example, can only be rational), which flatters the overall figures. On the remaining 55.7%, the cases that need judgement, accuracy is 78.3%.

Accuracy is 89.5% in normal markets, 87.8% in bull markets and corrections, 90.3% in euphoria and 77.6% in crashes, where loss aversion and herd behavior overlap the most. By action it is 84.9% for sales, 90.0% for holds and 89.4% for purchases. Trained only on decisions dated 2008 to 2019 and tested on 2020 to 2026, which includes the COVID crash, the model reaches 86.5% accuracy and 0.826 macro-F1, close to the random-split result.

7.2.1 Baselines and Other Learners

Model (real-frequency test set)	**Accuracy**	**Macro-F1**
Always "rational"	57.5%	–
Previous model v3 (same test cases)	53.4%	0.309
Definitions only (call a bias whenever one can apply)	85.2%	0.825
v4.1 without the `allows` inputs	87.6%	0.832
v4.1	**87.9%**	**0.840**
Ceiling: v4.1 also given the hidden tendencies	90.0%	0.866

Table 7.5: Baselines, ablation and ceiling

Table 7.5 is important for reading the model honestly. Because the labels come from written definitions, a rules-only baseline already reaches 85.2%, and the model's gain over it is 2.7 points. The value of the model is in two places. It disagrees with the rules in 9.6% of cases and is then right 64% of the time against 36% for the rules,

because it recognizes rebalancing, a near goal, cutting a loss and a stated need for cash. It also gives calibrated confidence, which the rules cannot. The ceiling of 90.0% shows that about one case in ten cannot be resolved from what the app observes. The previous model v3 scored 53.4% on the same cases because it did not see the action: across 600 test cases, changing only the action changed v4.1's answer in 83.7% of them and v3's in none.

Model	Accuracy (5 seeds)	Accuracy, seed 7 [95% CI]	XGBoost minus model	McNemar p
Rules only	84.4% ± 0.6	85.2% [83.9, 86.4]	+2.7 [+1.6, +3.8]	2.4×10^{-7}
Logistic regression	84.9% ± 0.4	85.5% [84.2, 86.6]	+2.4 [+1.4, +3.4]	1.0×10^{-5}
Random forest	87.0% ± 0.5	87.6% [86.5, 88.7]	+0.2 [−0.4, +0.9]	0.096
XGBoost	87.2% ± 0.6	87.8% [86.7, 89.0]	–	–

Table 7.6: Learners compared on the same features and guard (5 split seeds; bootstrap CIs and McNemar tests on seed 7)

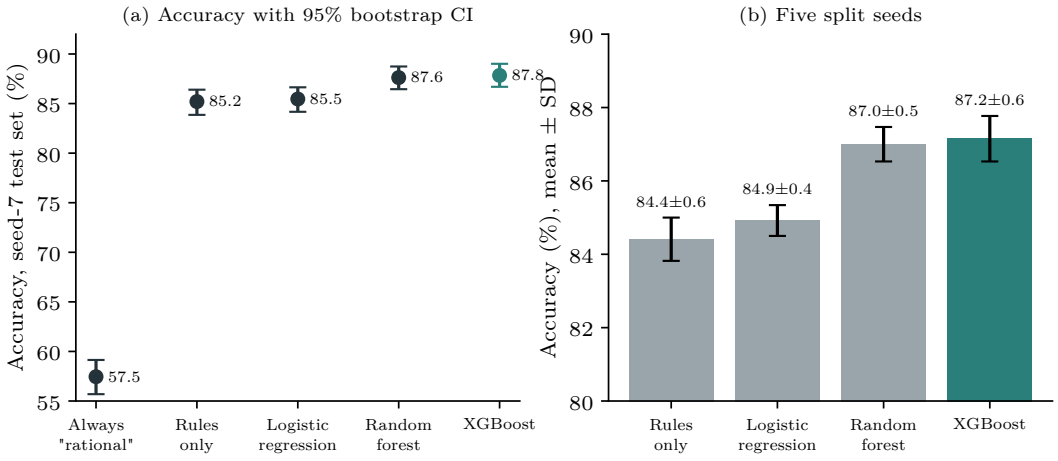


Figure 7.3: (a) Accuracy with 95% bootstrap confidence intervals; (b) mean and standard deviation over five split seeds

To check whether XGBoost is the right learner, logistic regression and a 500-tree random forest were trained on the same 22 features with the same guard and weighting, over five split seeds, with 2,000 bootstrap resamples and McNemar's test on the original split (Table 7.6, Figure 7.3). XGBoost is significantly better than the rules and logistic regression, which shows that the non-linear interactions matter. It is statistically tied with the random forest (a difference of 0.2 points with $p = 0.096$), so we do not claim that XGBoost is the better learner. It was kept because it has the lowest log-loss (0.267 against 0.282 for the forest), exact and fast TreeSHAP explanations and a portable model file.

Of the test cases, 167 are sales forced by money. Bias model v4.1 reads 71.9% of them as rational: 90.6% when the need is stated in the comment and 39.3% when it is not, against 37.7% for the earlier v4.0 model trained without money pressure. The gap between stated and unstated needs is exactly what the comment reader is designed to close. The training script also runs 13 scenario checks and 8 relational checks on a typical investor, and v4.1 passes all 21 (v3 passed 5 of the 13 scenarios). For example, selling a winner up 35% in a calm market reads as the disposition

effect with confidence 0.98, selling at a 20% loss in a calm market reads as rational, and in a fearful market a stated cash need raises the rational reading from 0.09 to 0.49. Gold share alone barely moves the reading (largest change 0.05), as it should, since gold has no role in the label definitions.

7.3 Simulator Results

7.3.1 Realism and Choice of Generator

Generator	1-yr p10	1-yr median	1-yr p90	ACF of squared returns, lag 1/3/6	KS distance
Real NIFTY (217 windows)	−7.5%	11.3%	33.3%	0.136 / 0.051 / 0.093	–
Lognormal (GBM)	−16.2%	9.5%	42.1%	−0.01 / −0.01 / −0.01	0.132
Independent months	−15.9%	10.6%	41.2%	−0.01 / −0.01 / −0.01	0.108
Block 3 months	−14.3%	10.6%	37.8%	0.077 / −0.01 / −0.01	0.083
Block 6 months	−13.7%	10.5%	35.9%	0.100 / 0.029 / −0.01	0.072
Block 12 months (used)	**−8.1%**	**10.3%**	**33.1%**	**0.101 / 0.035 / 0.025**	**0.027**
Block 24 months	−8.1%	10.3%	33.0%	0.105 / 0.023 / 0.033	0.030

Table 7.7: Six market path generators against the real index (10,000 ten-year paths each, same historical mean)

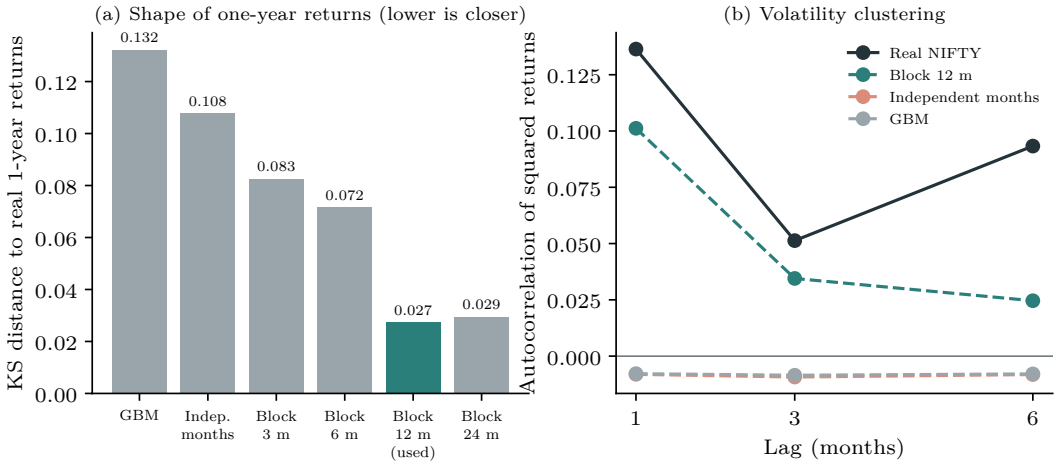


Figure 7.4: (a) Kolmogorov-Smirnov distance to real one-year returns; (b) volatility clustering

Table 7.7 and Figure 7.4 justify the choice of generator. All six generators share the same average growth, so only the shape and the dependence differ. The 12-month block bootstrap is the only one that reproduces both the spread of real one-year returns and volatility clustering. Its KS distance (0.027) is a quarter of that of independent months (0.108) and a fifth of GBM's (0.132), while GBM and independent months have no clustering at all and make bad years about twice as bad as reality. Moving to 24-month blocks changes almost nothing. With the long-run growth assumptions used in the app, simulated one-year returns are −6.9%, +11.6% and +34.7% at the 10th, 50th and 90th percentiles, against −7.5%, +11.3%

and +33.3% for the real index; the earlier v1 simulator had bad years of −15.8%. The 19-year record holds only one independent decade, so ten-year comparisons (KS distance about 0.34 for every generator) are not used to rank generators.

7.3.2 Effect of the Horizon and Stability of the Numbers

Horizon	Stock down 17%: holding wins	Typical gap	Fund up 30%: holding wins	Typical gap
1 year	55%	Rs. 3,592	66%	Rs. 6,377
3 years	60%	Rs. 13,950	70%	Rs. 22,244
5 years	59%	Rs. 21,124	70%	Rs. 41,640
10 years	62%	Rs. 59,925	75%	Rs. 1.09 L
20 years	64%	Rs. 1.81 L	80%	Rs. 4.09 L
30 years	61%	Rs. 3.49 L	81%	Rs. 13.9 L

Table 7.8: Effect of the horizon (Rs. 1 lakh position; investor earning Rs. 18 lakh a year with Rs. 11 lakh liquid)

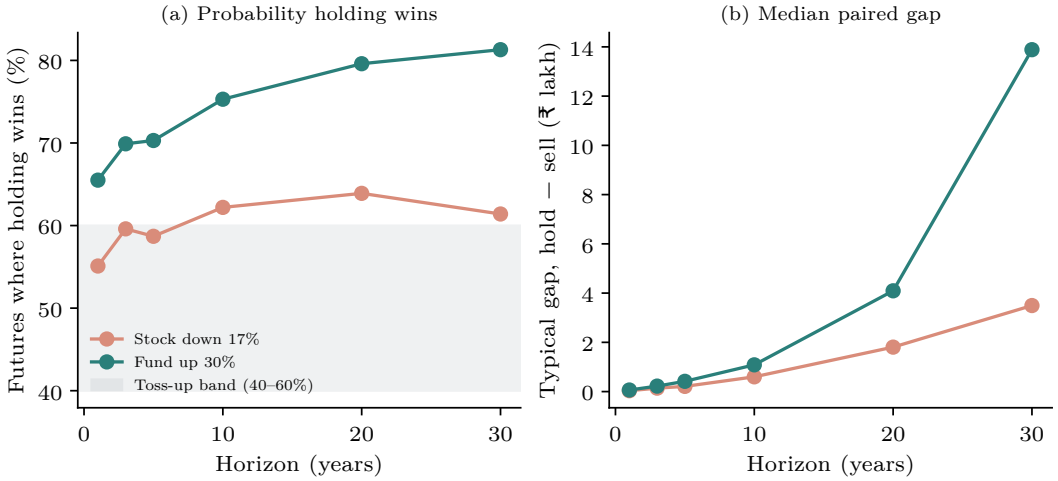


Figure 7.5: (a) Share of futures in which holding beats selling; (b) median paired gap, by horizon

Table 7.8 and Figure 7.5 show the central practical result of the simulator. For a single stock down 17%, the probability that holding beats selling stays between 55% and 64% at every horizon, mostly inside or near the band of 40 to 60% that the recommendation rules treat as a toss-up. A diversified fund, with little company-specific risk, favors holding much more clearly and reaches 80% at 20 years. The earlier simulator always used a 20-year horizon and therefore tilted every recommendation towards holding, even for someone who needs the money in three years.

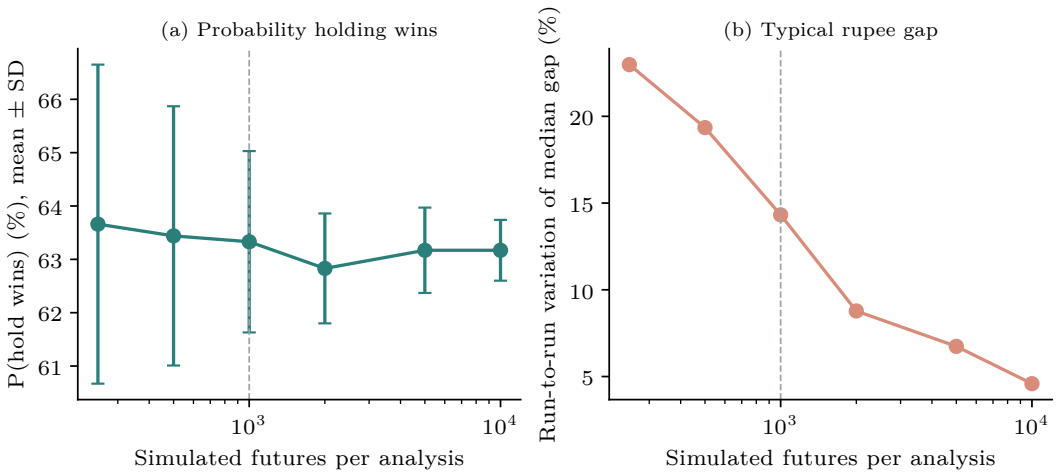


Figure 7.6: Monte Carlo convergence for the base case (20 repeats per size)

Figure 7.6 shows how much of each number is simulation noise. At the app's 1,000 futures, the probability that holding wins (the input to the recommendation rules) varies by about ±1.7 points between runs, close to the binomial standard error of ±1.5 points. The median rupee gap is noisier and varies by about ±14%. This is why the verdict is driven by the probability and the rupee gap is presented as an approximate value.

7.3.3 Results over 1,000 Investors and Sensitivity

Group	Investors	Mean P(hold wins)	HOLD / toss-up / SELL
All (long-run assumptions)	1,000	0.635	58.1% / 40.8% / 1.1%
Horizon 1 to 3 years	52	0.571	29% / 71% / 0%
Horizon 4 to 10 years	100	0.598	43% / 57% / 0%
Horizon 11 to 20 years	275	0.624	57% / 41% / 2%
Horizon 21 to 30 years	573	0.652	64% / 35% / 1%
Single stock	596	0.562	31% / 67% / 2%
Equity fund	404	0.742	99% / 1% / 0%
History replayed exactly	1,000	0.380	5.6% / 37.8% / 56.6%

Table 7.9: Paired simulation over 1,000 investors drawn from dataset v4.1

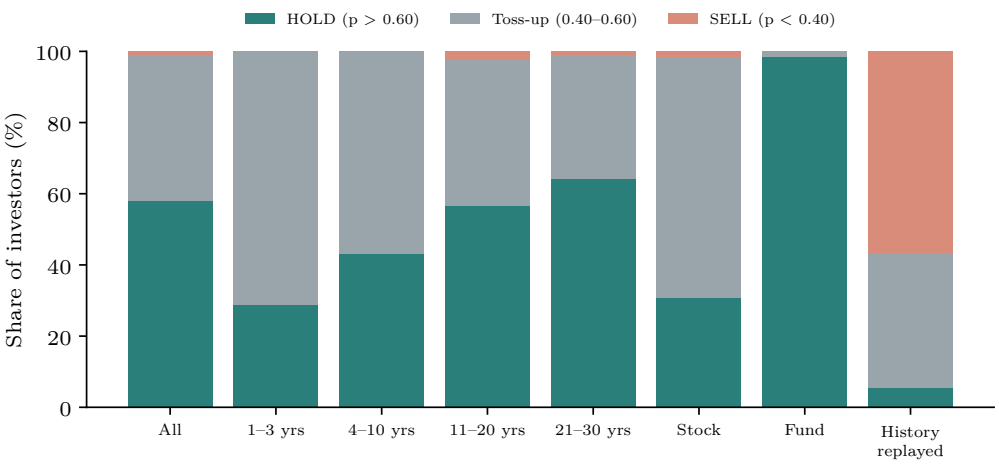


Figure 7.7: Share of investors in each verdict band

Table 7.9 and Figure 7.7 move from single examples to a population. Under the long-run assumptions, holding is clearly favored for 58% of investors, the comparison is a toss-up for 41% and selling is favored for 1%. Almost every equity fund (99%) clearly favors holding, while only 31% of single stocks do, because company-specific risk lowers the typical outcome. Whether the position is currently at a gain or a loss makes no difference (59% and 57% HOLD), which is exactly the point of the disposition effect literature: the purchase price should not matter for a forward-looking decision. The last row is the most important caution. If 2007 to 2026 is replayed exactly, gold compounds at about 15% a year instead of 9%, and 86% of verdicts change.

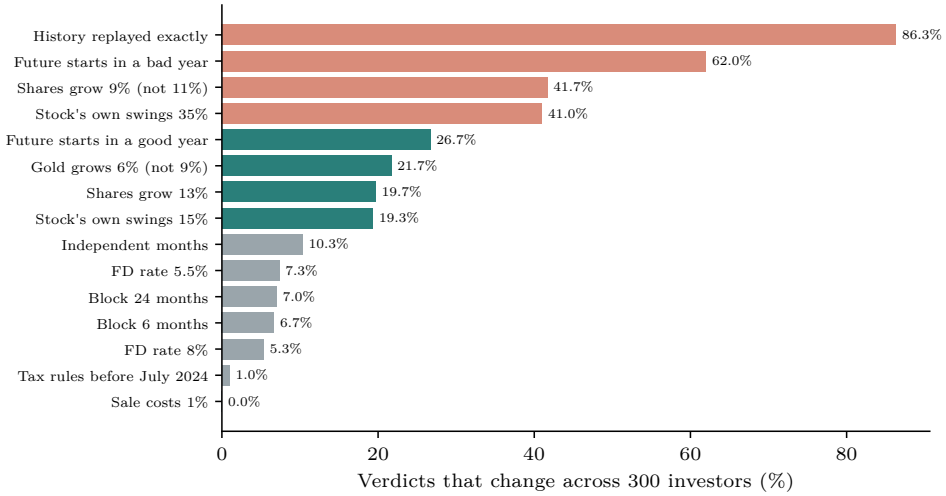


Figure 7.8: Share of verdicts that change across 300 investors when one assumption is changed

Figure 7.8 ranks the assumptions by how many verdicts they change. The answer depends most on the growth assumptions for shares and gold (shares growing at 9% instead of 11% changes 41.7% of verdicts), on how volatile the particular stock is (own volatility of 35% instead of 21% changes 41.0%) and on the kind of year the

future starts in (a bad first year changes 62.0%). It barely depends on the block length, the July 2024 tax change (1.0%) or sale costs (0%, since the cost is charged on cashing out in both futures). This is why Folium measures each stock's risk at submission instead of assuming it, and why the growth assumptions are stated on every result page.

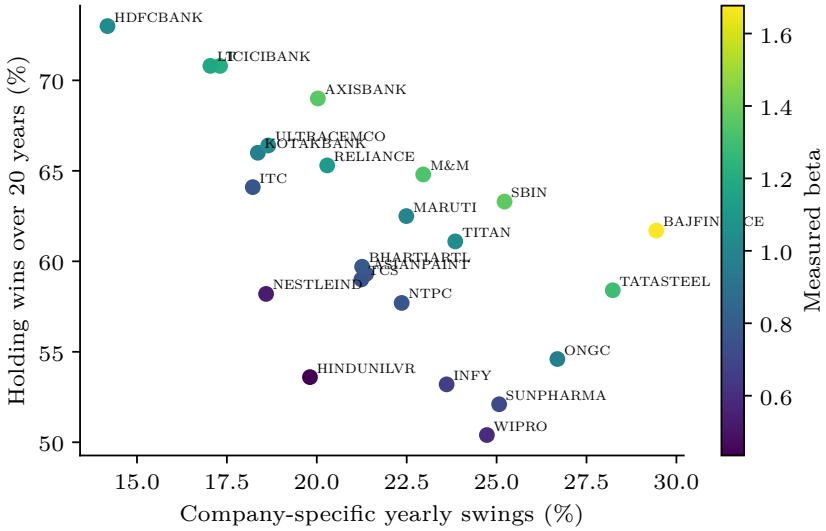


Figure 7.9: Company-specific volatility against the share of 20-year futures in which holding wins, for 24 NIFTY 50 stocks

Figure 7.9 shows a clear negative relationship between a stock's own volatility and the advantage of holding it. HDFC Bank, with beta 1.03 and own volatility of 14%, wins in 73% of 20-year futures, while Wipro, with beta 0.60 and own volatility of 25%, wins in 50%. Two effects combine: own volatility drags down the median compound outcome, and a low beta lowers expected growth under the capital asset pricing relation. The second effect is a modeling choice, since steady low-beta stocks have often done better historically than the relation predicts.

7.3.4 Comparison with Real History

Window	Real windows (independent)	Hold won, real	Hold won, simulated	Real median gap	Real inside simulated 10–90% band
5 years	169 (3)	88.8%	61.0%	Rs. 56,378	99.4%
10 years	109 (1)	100%	60.7%	Rs. 1,25,114	100%

Table 7.10: Walk-forward replay of a fund up 30% through every real window

Table 7.10 places what actually happened in each real 5-year and 10-year window inside the simulated distribution for the same case. Every real outcome but one falls inside the simulated 10th to 90th percentile band, and real outcomes sit in the upper middle of the band, since 2007 to 2026 was a good period for holding equity. With only one to three independent windows this is a consistency check and not a validation of forecasting ability. A related check concerns the paired gap itself: for the base-case investor over 14 years, the difference of the two medians

and the median of the paired differences differ by up to 19% at year 10 (Rs. 63,790 against Rs. 51,717). Folium reports the paired gap with its 10th to 90th percentile range (−Rs. 1,19,371 to Rs. 5,12,349 at year 10), because that range also shows that holding loses in a substantial share of futures even when it wins in the typical one.

7.4 Cause and Effect Results

Factor	Change	Sell	Hold	Buy more
A cash shock	having one vs not	+31.8 (+31.3)	−23.6 (−23.3)	−8.2 (−8.0)
Emergency fund	having one vs not	−7.5 (−5.6)	+1.4 (−0.1)	+6.1 (+5.7)
Market fear	VIX +10 points	+6.4 (+3.9)	−4.7 (−2.9)	−1.7 (−1.0)
Weaker NIFTY month	5 points worse	+5.6 (+5.8)	−3.9 (−3.7)	−1.7 (−2.1)
Bigger holding	+10 points of liquid	+4.8 (+4.7)	+2.7 (+2.8)	−7.5 (−7.5)
Longer horizon	+5 years	−2.4 (−2.3)	+1.4 (+1.4)	+1.0 (+1.0)
Gold (negative control)	+10 points	0.0 (−0.2)	0.0 (+0.4)	0.0 (−0.2)

Table 7.11: Average effects in percentage points: true value, with the g-computation estimate in brackets

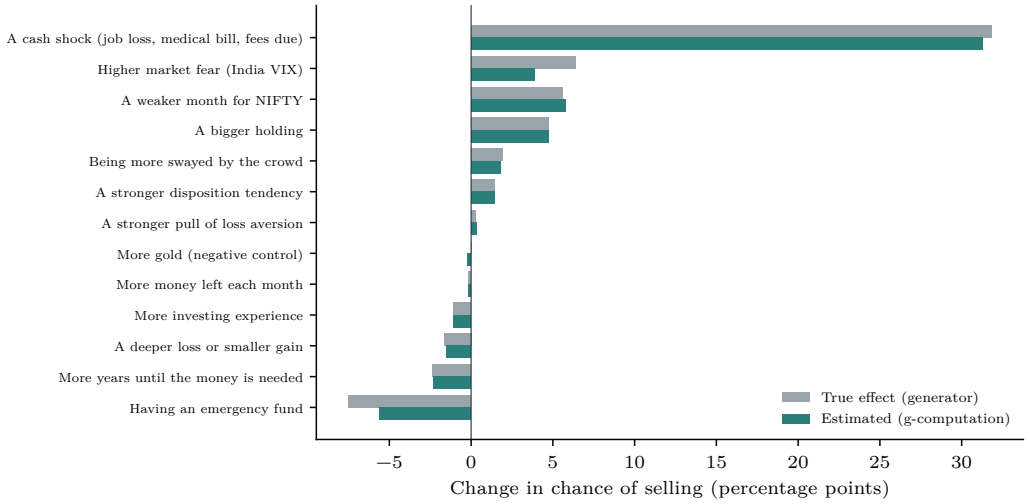


Figure 7.10: Effect of each factor on the chance of selling: true effect against the g-computation estimate

Table 7.11 and Figure 7.10 show that money pressure dominates. A cash shock raises the chance of selling by about 32 points, and an emergency fund lowers it by 7.5 points, mostly by absorbing shocks. Market fear and a weak month push towards selling, and a bigger holding pushes towards selling (largely through rebalancing) and strongly away from adding. Loss aversion's population-wide effect is small (+0.3 points) because it acts only in specific situations; within them it is large.

Across all 45 effects, the g-computation estimates have a mean absolute error of 0.48 points and the correct direction in 97% of cases. For effects on selling, g-computation's error is 0.45 points against 1.07 for the linear DoWhy model, which

for example overstates the effect of investing experience about fivefold. The linear model is better at the effect of a tendency within its own situation (0.3 points against 1.4). DoWhy passed 12 of 13 refutation tests; the one failure is the gold negative control, whose true effect is zero, so a test against chance cannot confirm anything. When the most helpful habit is applied in each situation, the estimates always move in the right direction and, where they are off, they understate the benefit. For example, setting selling rules in advance lowers the share of winners sold in a calm market from 56% to 39% in truth, and the estimate gives 54% to 40%. One unexpected finding is that habits can work against each other: for someone holding a deep loser, all three habits together barely help (43% to 42%), while reframing the loss alone helps a lot (43% to 36%). For this reason the app offers the single most helpful habit and never a bundle.

7.5 Explanation Quality

Explanation quality was measured on the current pipeline for 166 sell and hold decisions drawn from the population study, explained in both English and Hindi (332 explanations), with AI wording switched off. Every explanation was complete (all six sections present), actionable and personalized, and none contained a jargon term. The English explanations have a mean Flesch reading ease of 76.4 (minimum 73.5, against a target of 60), which corresponds to fairly easy text, with 11.3 words per sentence. The Hindi explanations average 12.8 words per sentence against a target of 20; Flesch is not used for Hindi because its syllable model is undefined for Devanagari. These are measures of structure and readability; they do not show that the explanations change investor behavior, which would need a user study.

7.6 Application Screenshots

The screenshots in this section are to be captured from the running application with the demonstration accounts created by `scripts/seed_demo.py`.

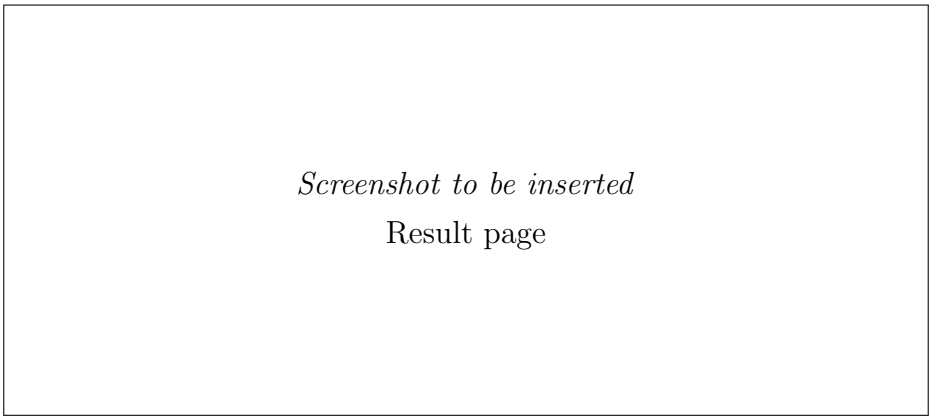


Figure 7.11: Result page with the recommendation, both futures and the pattern found

Screenshot to be inserted
Live progress

Figure 7.12: Live five-step progress during an analysis

Screenshot to be inserted
Hindi explanation

Figure 7.13: Six-part explanation in Hindi

Screenshot to be inserted
Portfolio page

Figure 7.14: Portfolio page with net worth, allocation and holdings

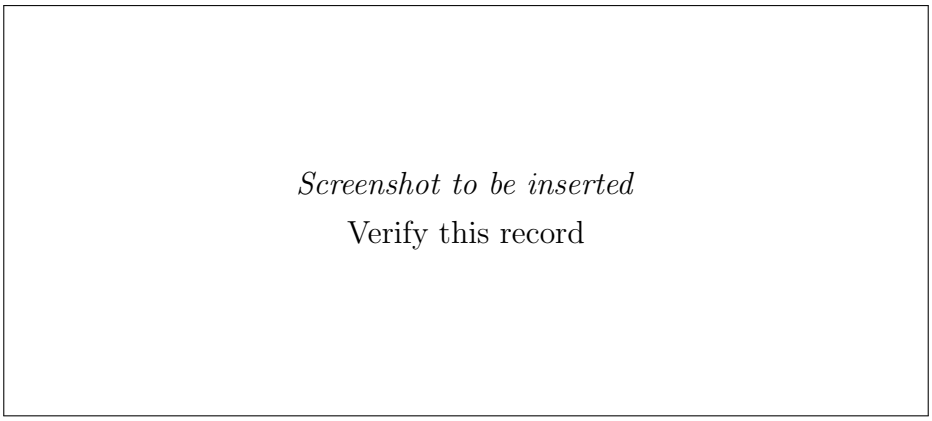


Figure 7.15: Verification of an audit record

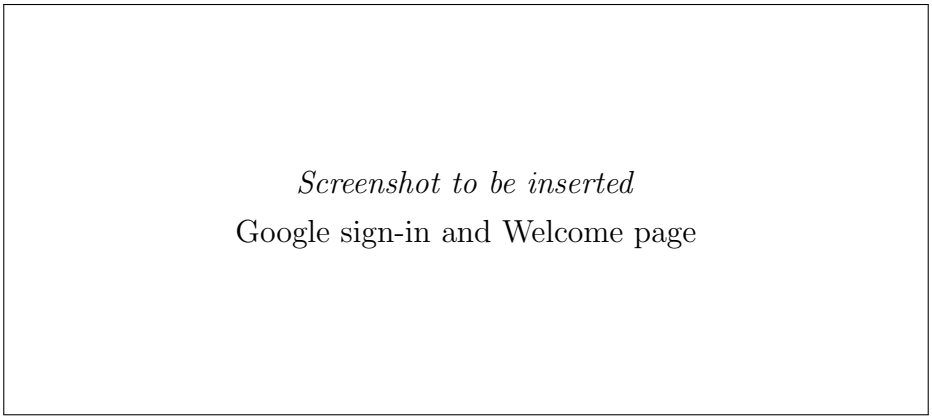


Figure 7.16: Google sign-in return and the Welcome profile step

7.7 Discussion

On the development task, each analytical component does what it was designed to do. The classifier is accurate, calibrated and consistent with the label definitions, significantly better than the rules and a linear model and on par with a random forest. It changes its reading when the action changes, recognizes forced sales when the need is stated and generalizes across time periods within the data. The simulator's generator was chosen experimentally against five alternatives. It reproduces the real index's one-year distribution and volatility clustering, its probability output is stable at the app's path count, and every real window falls inside its band. The causal estimates recover a known mechanism closely and never overclaim the benefit of habits, and the explanations are complete, readable and free of jargon on the current models.

The results also have clear boundaries. The behavioral labels are synthetic, so 87.9% accuracy means that the model recovers the generator's labels, not that it diagnoses real investors, and most of that accuracy follows from the definitions themselves. The simulator's agreement with history is a distributional check and not a forecast, and its verdicts depend heavily on the stated growth assumptions. The causal results describe the generator and not people, and no study has measured whether investors who use Folium decide differently.

The combination is more useful than any part alone. For a typical request to sell a falling stock, the simulator often lands in the toss-up band (two-thirds of single-stock positions do), and the behavioral reading then decides whether waiting is the cautious choice. A stated need for cash moves the reading to rational and lets the sale stand. The investor sees why, in their own language, with every figure traceable, and can later check that the record has not been changed.

Chapter 8

Testing

Testing in Folium had four goals: to show that each model and engine behaves as specified, including behavioral properties and not only code paths; to show that the security controls block what they are meant to block; to show that the pipeline, database and live stream work together; and to prevent regressions when models are retrained or replaced.

8.1 Test Suites

Suite	Location	Tests	Needs
Models and engines	`tests/`	343	Python environment (5 tests need an AI key)
Back end, full	`backend/tests/`	267	PostgreSQL and Redis (149 of these run without a database)
Front end	`frontend/src/test/`	25	Node.js (Vitest, Testing Library)

Table 8.1: Automated test suites (635 tests in total)

The counts in Table 8.1 are those recorded in the project's master reference after the latest changes, where all suites were reported as passing. Many tests are parameterized, so one test function runs over several cases.

8.2 Unit Testing

ID	Test	Property checked
UT-1	Training and live features agree	One row and a batch build identical features
UT-2	Invalid input rejected	A missing input or unknown action raises an error
UT-3	Label rules match the dataset	The true label is allowed for all 20,000 rows
UT-4	Guard and calibration	Ruled-out classes are removed; calibration never changes the chosen class
UT-5	SHAP drivers	Drivers sum to the chosen class's score
UT-6	Action sensitivity	The same situation with a different action gives a different reading
UT-7	Tax rules	20% short-term tax with cess; Rs. 1.25 lakh exemption; losses pay nothing
UT-8	Simulator behavior	Horizon clamped to 1 to 30 years; same inputs give the same answer; SIP leaves the gap unchanged; simulated years resemble the real NIFTY
UT-9	Recommendation rules	Every combination of intent, pattern and p; a missing simulation is disclosed
UT-10	Audit chain	An edited, removed or forged entry and a fork are all caught; fingerprints survive database round trips
UT-11	Comment reading	Rules in three languages; quotes are verbatim; both-sides cases never alter the verdict
UT-12	Explanations	Six sections, English and Hindi structural parity, jargon blocklist, number check

Table 8.2: Representative unit tests

8.3 Integration Testing

Integration tests run the FastAPI application against real PostgreSQL and Redis instances through an HTTP client.

ID	Scenario	Expected result
IT-1	Register, verify, log in, refresh, log out	Cookies set and rotated; old tokens blacklisted
IT-2	Enable TOTP; log in with a code and a backup code	Second factor enforced; backup code single-use
IT-3	Mutating request with missing, wrong and correct CSRF header	403, 403, accepted; login exempt
IT-4	User B queries user A's rows; query with no user set	No rows in either case
IT-5	Repeated logins beyond the limit	HTTP 429
IT-6	Submit an analysis and run the pipeline	202; status moves to processing and then complete
IT-7	Open the stream during and after a run	Backlog replayed without duplicates; final event received
IT-8	Two analyses, then tamper with a row and verify	Chain stays linear; the changed part is named
IT-9	Reason says "college fees are due"	Cash need detected; sale can read as rational
IT-10	Delete, log in during grace, purge after grace	"Keep my account" offered; data anonymized; audit kept
IT-11	Google sign-in with a stand-in for Google	Correct redirect for each outcome; unverified addresses refused; state single-use
IT-12	Google return page for each outcome (front end)	Signs in, asks for the code, offers to keep the account or explains the failure

Table 8.3: Integration test scenarios

8.4 System Testing

The behavior of the product as a whole is exercised through the demonstration walkthrough, which uses accounts and analyses created by the seeding script. Table 8.4 lists the system test cases with their expected results.

ID	Test case	Expected result
ST-1	Sell a losing holding with the reason “need money for college fees in March”	Pattern reads rational and the quote is shown
ST-2	Sell a holding up 30% in a calm market	Disposition effect with High or Medium confidence
ST-3	Buy more after a NIFTY rise of 5% in a month	Herd reading and a HOLD verdict
ST-4	Switch the explanation to Hindi	All six sections in Hindi with the same numbers
ST-5	Request a Hindi PDF report	Refused with an explanation; HTML offered
ST-6	Reload the page during step 2	Earlier steps shown from the backlog; progress continues
ST-7	Verify a record, then edit it in the database and verify again	“Verified”, then “Changed” with the changed part named
ST-8	Run an analysis with market data switched off	Calm market assumed and disclosed
ST-9	First sign-in with a new Google account	Returns to the app; Welcome page asks for the profile

Table 8.4: System test cases

8.5 Validation, Edge Cases and Performance

Input is validated at three layers. Pydantic schemas reject malformed requests with HTTP 422 before any logic runs. Database check constraints enforce domain rules whatever the code path, for example that age lies between 18 and 100, that the quantity is positive, that confidence lies between 0 and 1, and that a recommendation is one of HOLD, SELL or BUY MORE. The model and the simulator raise an error on missing or invalid inputs. The edge cases of Table 6.3 are covered by tests such as horizon clamping, the missing-simulation rule and the price outage.

No load test has been performed. One simulation of 1,000 paths over 30 years takes 103.4 ms, and each prediction records its own inference time in milliseconds. Numerical stability, which matters more for the correctness of a verdict, was measured instead: at 1,000 paths the probability that holding wins varies by only about ± 1.7 points between runs.

8.6 Defects Found and Resolved

#	Issue	Status
1	The application connected as a superuser, which bypasses row-level security	Fixed: connects as `app_user`
2	v3's explanation described a different class from the one shown (98.6% of cases)	Fixed in v4
3	The Google sign-in callback returned JSON instead of returning to the app	Fixed: redirect and Welcome page
4	The slab rate on interest was computed from take-home pay	Fixed: income grossed up
5	A driver label read "Your choice to holding on"	Fixed: separate verb mapping
6	The fallback story always said "over 20 years"	Fixed: uses the actual horizon
7	The explanation evaluation had been run on earlier models	Fixed: re-measured on 332 explanations
8	The AI wording check rejects added numbers but not removed ones	Open
9	Some column names still end in `_20y` although the horizon is variable	Open (cosmetic)

Table 8.5: Defects and their status

Chapter 9

Conclusion and Future Scope

9.1 Conclusion

Retail investors often decide to sell, hold or buy more under the pull of recent losses, market fear or the crowd, while the tools available to them describe the asset rather than the decision. In this project we built Folium, a full-stack web application that judges one intended decision against the investor's whole financial picture. For each decision, a five-step background pipeline reads the investor's comment, classifies the behavioral pattern with a calibrated and guarded XGBoost model explained by exact TreeSHAP, projects holding and selling on the same block-bootstrapped market paths after Indian tax, attaches causal estimates validated against a known generator, renders a six-part explanation in English and Hindi, derives a verdict from explicit evidence rules, and seals the analysis in a per-investor hash chain. Investors sign in by e-mail or with Google, accounts are protected with two-step verification, CSRF protection and rate limits, and investors can export or delete their data.

Objective	**Outcome**
Causal decision-time framework for detecting behavioral biases	87.9% accuracy, macro-F1 0.840 and ECE 0.014; High readings right 98.7% of the time; 21 of 21 sense checks; 86.5% on an unseen period; 45 causal effects recovered with 0.48 points mean error and the correct direction in 97% of cases
Personalized portfolio-level long-horizon simulation	Horizon of 1 to 30 years, after tax, on paired paths; one-year returns within 1.4 points of the real NIFTY; best of six generators (KS 0.027); probability stable to ±1.7 points at 1,000 paths
Decision-centric counterfactual explanations	Six-part explanations in English and Hindi with year-by-year projections of both choices; 332 explanations complete and jargon-free; English Flesch reading ease 76.4
Tamper-proof audit mechanism	SHA-256 hash chain for every analysis with verification that detects altered, removed and forged entries and forks; append-only table; per-investor locking

Table 9.1: Achievement of the objectives

Table 9.1 summarizes how each objective was met. All figures are development-stage results on synthetic behavioral labels.

9.2 Limitations

Two boundaries define what the results can claim. The first is scientific. The behavioral labels are synthetic by design, because no public dataset links Indian investors' decisions to their finances and true motives. The readings are therefore validated against a known generator and not against real people, and no claim is made that the system improves investment outcomes. The second concerns modeling. The simulator's verdicts depend on stated long-run growth rates, projections are in nominal rupees, tax handling is simplified (losses are not set off against other gains), mutual funds are treated as equity funds, and each analysis concerns one position. Both boundaries are stated to the investor on every result page.

9.3 Future Scope

The remaining work is organized in three phases, each building on foundations that already exist in the repository.

Phase	Milestone	Foundation already in place
Productization	Administrator console (usage statistics, user management, recovery queue)	Aggregate statistics view without personal data; audited two-step reset endpoints; admin role
	Continuous integration	635 tests that run with one command per suite; pinned dependencies
	Public deployment with HTTPS and production e-mail	Container definitions; secure cookies and CORS outside development; e-mail integration
	Live AI assistance	Consent switch, verbatim-quote and number checks, circuit breaker, all tested with a stand-in
Validation with real users	Consented real decisions, so that motive weights can be learned	Data export and consent model; outcome-tracking columns in the decisions table
	Prospective user study	Explanation engine with measured readability
	Two-way number check for AI wording	Existing check and its tests
Richer analysis	Inflation-adjusted output, debt and hybrid funds, loss set-off, other uses of sale proceeds	Evaluation hooks in the simulator; sensitivity framework
	Whole-portfolio decisions	Portfolio snapshot and asset-mix computation
	More Indian languages; background job queue and load testing	Bilingual template banks with parity tests; stateless API with shared state in Redis

Table 9.2: Future scope by phase

The first phase turns a complete local product into a public service, the second turns development-stage evidence into evidence about people, and the third widens what the system can analyze. Longer-term ideas from the original plan, such as broker integration, alerts and periodic retraining from tracked outcomes, follow naturally once real outcome data is available. Until then, Folium should be presented exactly as it presents itself: as information and education, and not as investment advice.

Bibliography

[1] D. Kahneman and A. Tversky, "Prospect theory: An analysis of decision under risk," *Econometrica*, vol. 47, no. 2, pp. 263–291, 1979, doi: 10.2307/1914185.

[2] T. Odean, "Are investors reluctant to realize their losses?" *The Journal of Finance*, vol. 53, no. 5, pp. 1775–1798, 1998, doi: 10.1111/0022-1082.00072.

[3] S. Bikhchandani, D. Hirshleifer and I. Welch, "A theory of fads, fashion, custom, and cultural change as informational cascades," *Journal of Political Economy*, vol. 100, no. 5, pp. 992–1026, 1992, doi: 10.1086/261849.

[4] H. Shefrin and M. Statman, "The disposition to sell winners too early and ride losers too long: Theory and evidence," *The Journal of Finance*, vol. 40, no. 3, pp. 777–790, 1985, doi: 10.1111/j.1540-6261.1985.tb05002.x.

[5] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[6] B. Zadrozny and C. Elkan, "Transforming classifier scores into accurate multiclass probability estimates," in *Proc. 8th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2002, pp. 694–699, doi: 10.1145/775047.775151.

[7] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 4765–4774.

[8] M. T. Ribeiro, S. Singh and C. Guestrin, "Why should I trust you?: Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD*, 2016, pp. 1135–1144, doi: 10.1145/2939672.2939778.

[9] S. Wachter, B. Mittelstadt and C. Russell, "Counterfactual explanations without opening the black box: Automated decisions and the GDPR," *Harvard Journal of Law & Technology*, vol. 31, no. 2, pp. 841–887, 2018.

[10] J. Pearl, *Causality: Models, Reasoning, and Inference*, 2nd ed. Cambridge, U.K.: Cambridge University Press, 2009.

[11] A. Sharma and E. Kiciman, "DoWhy: An end-to-end library for causal inference," arXiv:2011.04216, 2020.

[12] H. R. Künsch, "The jackknife and the bootstrap for general stationary observations," *The Annals of Statistics*, vol. 17, no. 3, pp. 1217–1241, 1989, doi: 10.1214/aos/1176347265.

[13] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall, 1993.

[14] B. M. Barber and T. Odean, "Trading is hazardous to your wealth: The common stock investment performance of individual investors," *The Journal of Finance*, vol. 55, no. 2, pp. 773–806, 2000, doi: 10.1111/0022-1082.00226.

[15] B. M. Barber and T. Odean, "Boys will be boys: Gender, overconfidence, and common stock investment," *The Quarterly Journal of Economics*, vol. 116, no. 1, pp. 261–292, 2001, doi: 10.1162/003355301556400.

[16] M. Grinblatt and M. Keloharju, "What makes investors trade?" *The Journal of Finance*, vol. 56, no. 2, pp. 589–616, 2001, doi: 10.1111/0022-1082.00338.

[17] A. Tversky and D. Kahneman, "Advances in prospect theory: Cumulative representation of uncertainty," *Journal of Risk and Uncertainty*, vol. 5, pp. 297–323, 1992, doi: 10.1007/BF00122574.

[18] R. Thaler, "Mental accounting and consumer choice," *Marketing Science*, vol. 4, no. 3, pp. 199–214, 1985, doi: 10.1287/mksc.4.3.199.

[19] J. B. De Long, A. Shleifer, L. H. Summers and R. J. Waldmann, "Noise trader risk in financial markets," *Journal of Political Economy*, vol. 98, no. 4, pp. 703–738, 1990, doi: 10.1086/261703.

[20] E. F. Fama, "Efficient capital markets: A review of theory and empirical work," *The Journal of Finance*, vol. 25, no. 2, pp. 383–417, 1970, doi: 10.2307/2325486.

[21] H. Markowitz, "Portfolio selection," *The Journal of Finance*, vol. 7, no. 1, pp. 77–91, 1952, doi: 10.2307/2975974.

[22] R. C. Merton, "Lifetime portfolio selection under uncertainty: The continuous-time case," *The Review of Economics and Statistics*, vol. 51, no. 3, pp. 247–257, 1969, doi: 10.2307/1926560.

[23] J. D. Hamilton, "A new approach to the economic analysis of nonstationary time series and the business cycle," *Econometrica*, vol. 57, no. 2, pp. 357–384, 1989, doi: 10.2307/1912559.

[24] M. López de Prado, *Advances in Financial Machine Learning*. Hoboken, NJ, USA: Wiley, 2018.

[25] L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[26] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001, doi: 10.1214/aos/1013203451.

[27] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in *Proc. 22nd Int. Conf. Machine Learning (ICML)*, 2005, pp. 625–632, doi: 10.1145/1102351.1102430.

[28] C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. 34th Int. Conf. Machine Learning (ICML)*, PMLR 70, 2017, pp. 1321–1330.

[29] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLOS ONE*, vol. 10, no. 3, e0118432, 2015, doi: 10.1371/journal.pone.0118432.

[30] R. K. Mothilal, A. Sharma and C. Tan, "Explaining machine learning classifiers through diverse counterfactual explanations," in *Proc. Conf. Fairness, Accountability, and Transparency (FAT*)*, 2020, pp. 607–617, doi: 10.1145/3351095.3372850.

[31] D. Janzing, L. Minorics and P. Blöbaum, "Feature relevance quantification in explainable AI: A causal problem," in *Proc. 23rd Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, PMLR 108, 2020, pp. 2907–2916.

[32] V. Chernozhukov, D. Chetverikov, M. Demirer, E. Duflo, C. Hansen, W. Newey and J. Robins, "Double/debiased machine learning for treatment and structural parameters," *The Econometrics Journal*, vol. 21, no. 1, pp. C1–C68, 2018, doi: 10.1111/ectj.12097.

[33] D. N. Politis and J. P. Romano, "The stationary bootstrap," *Journal of the American Statistical Association*, vol. 89, no. 428, pp. 1303–1313, 1994, doi: 10.1080/01621459.1994.10476870.

[34] P. Weber, K. V. Carl and O. Hinz, "Applications of explainable artificial intelligence in finance: A systematic review of finance, information systems, and computer science literature," *Management Review Quarterly*, vol. 74, pp. 867–907, 2024, doi: 10.1007/s11301-023-00320-0.

[35] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal and S.-I. Lee, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, pp. 56–67, 2020, doi: 10.1038/s42256-019-0138-9.

[36] J. Robins, "A new approach to causal inference in mortality studies with a sustained exposure period: Application to control of the healthy worker survivor effect," *Mathematical Modelling*, vol. 7, no. 9–12, pp. 1393–1512, 1986, doi: 10.1016/0270-0255(86)90088-6.

[37] M. A. Hernán and J. M. Robins, *Causal Inference: What If*. Boca Raton, FL, USA: Chapman & Hall/CRC, 2020.

[38] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015, doi: 10.17487/RFC7519.

[39] D. M'Raihi, S. Machani, M. Pei and J. Rydell, "TOTP: Time-Based One-Time Password Algorithm," IETF RFC 6238, May 2011, doi: 10.17487/RFC6238.

[40] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annual Technical Conference (FREENIX Track)*, 1999, pp. 81–91.

[41] D. L. Wheeler, "zxcvbn: Low-budget password strength estimation," in *Proc. 25th USENIX Security Symposium*, 2016, pp. 157–173.

[42] OWASP Foundation, "Cross-Site Request Forgery Prevention Cheat Sheet," OWASP Cheat Sheet Series. [Online]. Available: `https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html`

[43] The PostgreSQL Global Development Group, "Row Security Policies," *PostgreSQL Documentation*. [Online]. Available: `https://www.postgresql.org/docs/current/ddl-rowsecurity.html`

[44] S. Haber and W. S. Stornetta, "How to time-stamp a digital document," *Journal of Cryptology*, vol. 3, no. 2, pp. 99–111, 1991, doi: 10.1007/BF00196791.

[45] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, Aug. 2015, doi: 10.6028/NIST.FIPS.180-4.

[46] WHATWG, "Server-sent events," *HTML Living Standard*. [Online]. Available: `https://html.spec.whatwg.org/multipage/server-sent-events.html`

[47] A. P. Oron and N. Flournoy, "Centered isotonic regression: Point and interval estimation for dose-response studies," *Statistics in Biopharmaceutical Research*, vol. 9, no. 3, pp. 258–267, 2017, doi: 10.1080/19466315.2017.1286256.

[48] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[49] R. Aroussi, "yfinance: Download market data from Yahoo! Finance's API," open-source library. [Online]. Available: `https://github.com/ranaroussi/yfinance`

[50] MFAPI.in, "Mutual Fund API for India." [Online]. Available: `https://www.mfapi.in`

[51] NSE Indices Ltd., "NIFTY 50 Index," and National Stock Exchange of India, "India VIX." [Online]. Available: `https://www.niftyindices.com`, `https://www.nseindia.com`

[52] Government of India, *The Finance (No. 2) Act, 2024* (Act No. 15 of 2024), provisions on capital gains tax rates and the long-term capital gains exemption.

[53] Government of India, *The Finance Act, 2025*, provisions on new-regime income tax slabs and the Section 87A rebate.

[54] Pallets Projects, “Jinja,” official documentation. [Online]. Available: https://jinja.palletsprojects.com

Chapter 10

Publications